UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI
DEPARTMENT OF INFORMATION AND COMMUNICATION TECHNOLOGY

Research and Development

GROUP PROJECT REPORT

By

Student group number 5

Vu Thi Kim Oanh – Student ID 23BA14225

Nguyen Khai Minh – Student ID 2410607

Huynh Kim Gia Bao – Student ID 2410154

Dang Nhat Minh – Student ID 2410667

Dao Chi Trung – Student ID 23BA14295

Title:

Time Series Analysis of Hanoi Garment CPI Data and Indonesia Economic Data

Hanoi, October 2026

Introduction

This report applies time series analysis with machine learning in Python to two data sets assigned to group 5. Part 1 studies the monthly consumer price index (CPI) of the group "Garments, footwear and hats" in Hanoi from January 2015 to December 2022 (file Data01_05_GarmentCPI.csv): we describe the series, check trend, autocorrelation and stationarity, identify and estimate ARIMA models, forecast the mean values and evaluate ARIMA and SARIMA models with a train/test split. Part 2 studies annual economic indicators of Indonesia from 1990 to 2021 (file Data02_05_Indonesia_C.csv) and detects anomalies (Z-score, Isolation Forest) and change points (PELT). All computations were done in Python 3.12 with pandas, statsmodels 0.15, scikit-learn and ruptures; the complete runnable notebook is Group5_TimeSeries.ipynb.

Part 1. Time Series Analysis of Hanoi CPI Data

Summary: We analyse the Hanoi garment, footwear and hat CPI and its monthly rate. Both series are stationary (d = 0). ARIMA(1,0,1) is selected for the CPI and ARIMA(0,0,1) for the CPI rate, by AIC and by test RMSE. Seasonal SARIMA terms do not improve the forecasts.

1.1 Introduction

Time series analysis methodology

A time series is a sequence of observations ordered in time. Time series analysis describes the series (plot, trend, seasonality), measures its dependence on its own past with the autocorrelation function (ACF) and partial autocorrelation function (PACF), and checks stationarity (constant mean, variance and autocovariance) with the Augmented Dickey-Fuller (ADF) and KPSS tests. The Box-Jenkins approach then builds an ARIMA(p,d,q) model: d is the number of differences needed for stationarity, p the order of the autoregressive part (suggested by the PACF cut-off) and q the order of the moving-average part (suggested by the ACF cut-off). Candidate models are estimated by maximum likelihood and compared with the Akaike Information Criterion (AIC). In the machine-learning view, the series is split into a training part and a test part, the model is fitted on the training data and evaluated by the root mean squared error (RMSE) of its forecasts on the test data. SARIMA(p,d,q)(P,D,Q,m) adds seasonal AR and MA terms with period m (m = 12 for monthly data).

CPI concept

The consumer price index measures the average change over time of the prices paid by households for a fixed basket of consumer goods and services. In Vietnam the General Statistics Office (GSO) publishes the CPI monthly for 11 main groups of goods and services; "Garments, footwear and hats" is one of them. An index value above 100 means prices are higher than in the base (reference) period, below 100 means they are lower. The CPI rate is the

percentage change of prices relative to the previous period and is the usual measure of inflation.

Data array "Data01_05_GarmentCPI.csv"

The file contains 96 monthly observations (January 2015 - December 2022) of the CPI of garments, footwear and hats in Hanoi. There are no missing values.

+ Column 1: Time - timestamp index 1..96
+ Column 2: Year (2015-2022)
+ Column 3: Month (1-12)
+ Column 4: Series 1 - GarmentFWHattoDecCPI: monthly garment, footwear and hat CPI with December of the previous year = 100 (mean 100.14, range 99.44-100.70)
+ Column 5: Series 2 - CPI_toPM: monthly CPI relative to the previous month (= 100 x Series 1(t) / Series 1(t-1); in January it equals Series 1, whose base is the previous December)
+ Column 6: Series 3 - CPI_rate: monthly CPI rate in % = Series 2 - 100 (mean 0.023 %, range -0.59 % to +0.70 %)

Following the template, the analysis is done for Series 1 (CPI) and Series 3 (CPI rate).

Code

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.stattools import acf, pacf, adfuller, kpss
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
from sklearn.ensemble import IsolationForest
import ruptures as rpt

plt.rcParams["figure.figsize"] = (10, 4)
plt.rcParams["figure.dpi"] = 110

cpi = pd.read_csv("GrProjCPIData/Data01_05_GarmentCPI.csv")
cpi.columns = cpi.columns.str.strip()
cpi.index = pd.date_range("2015-01-01", periods=len(cpi), freq="MS")
s1 = cpi["GarmentFWHattoDecCPI"].rename("CPI (series 1)")
s3 = cpi["CPI_rate"].rename("CPI rate (series 3)")
SERIES = {"CPI (series 1)": s1, "CPI rate (series 3)": s3}
print(cpi.head())
print(cpi.shape)
print(cpi[["GarmentFWHattoDecCPI", "CPI_toPM", "CPI_rate"]].describe().round(3))
```

Result

```
            Time  Year  Month  GarmentFWHattoDecCPI    CPI_toPM  CPI_rate
2015-01-01     1  2015      1                 100.7  100.700000  0.700000
2015-02-01     2  2015      2                 100.2   99.503476 -0.496524
2015-03-01     3  2015      3                 100.1   99.900200 -0.099800
2015-04-01     4  2015      4                 100.3  100.199800  0.199800
2015-05-01     5  2015      5                 100.3  100.000000  0.000000
(96, 6)
       GarmentFWHattoDecCPI  CPI_toPM  CPI_rate
count                96.000    96.000    96.000
mean                100.143   100.023     0.023
std                   0.196     0.214     0.214
min                  99.440    99.410    -0.590
25%                 100.060    99.900    -0.100
50%                 100.100   100.000     0.000
75%                 100.222   100.105     0.105
max                 100.700   100.700     0.700
```

1.2 Time series description

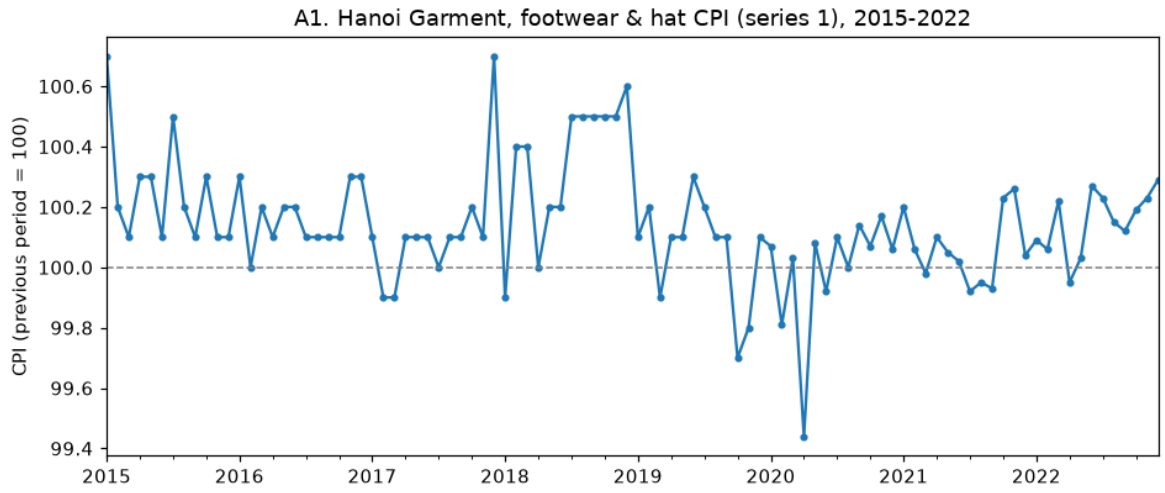
A. Time series plot

A1. CPI (series 1)

Code

```
s1.plot(marker="o", ms=3, title="A1. Hanoi Garment, footwear & hat CPI (series 1), 2015-2022")
plt.axhline(100, color="grey", ls="--", lw=1)
plt.ylabel("CPI (previous period = 100)")
plt.show()
```

Result



Comment on the result

The CPI moves in a narrow band between 99.44 and 100.70 around a mean of 100.14, and is above 100 in most months, i.e. garment prices in Hanoi are usually slightly higher than in the previous December. The highest values are

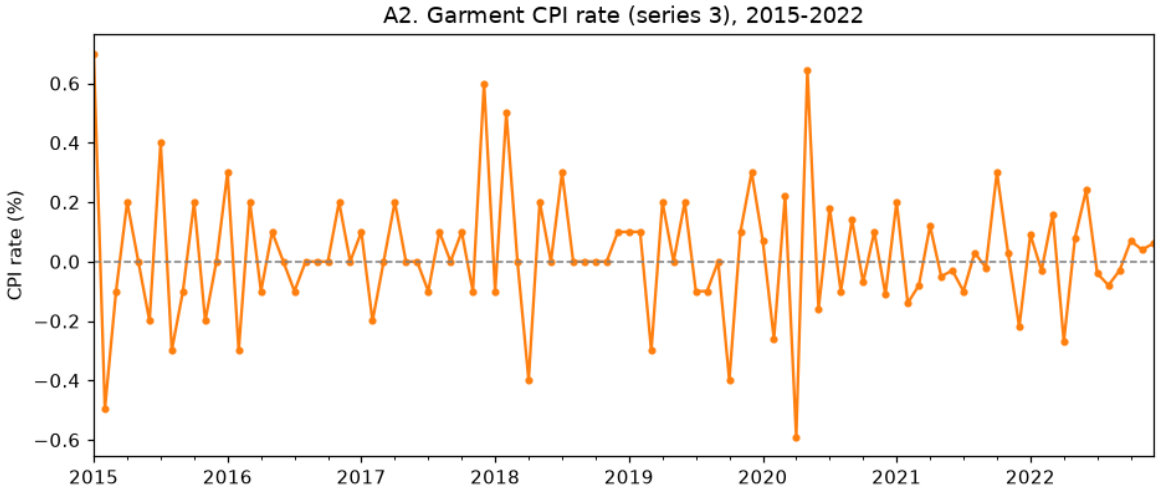
January 2015 and December 2017 (100.7, before the Lunar New Year), followed by a plateau at 100.5-100.6 from July to December 2018. The lowest value is April 2020 (99.44), the month of the nationwide COVID-19 social distancing, and values below 100 also appear in late 2019 and early 2020. From 2020 the values are recorded with two decimals and the fluctuations are smaller. The series has no explosive or long-run upward movement; it keeps returning to its mean, which suggests stationarity.

A2. CPI rate (series 3)

Code

```
s3.plot(marker="o", ms=3, color="tab:orange", title="A2. Garment CPI rate (series 3), 2015-2022")
plt.axhline(0, color="grey", ls="--", lw=1)
plt.ylabel("CPI rate (%)")
plt.show()
```

Result



Comment on the result

The CPI rate oscillates around zero (mean 0.023 %, standard deviation 0.214 %) between -0.59 % (April 2020) and +0.70 % (January 2015). Positive and negative months alternate frequently: a rise is often followed by a fall, which already suggests negative autocorrelation at lag 1. The variance is roughly constant, slightly smaller after 2020. There is no visible trend.

B. Time series trend checking

B1. CPI (series 1)

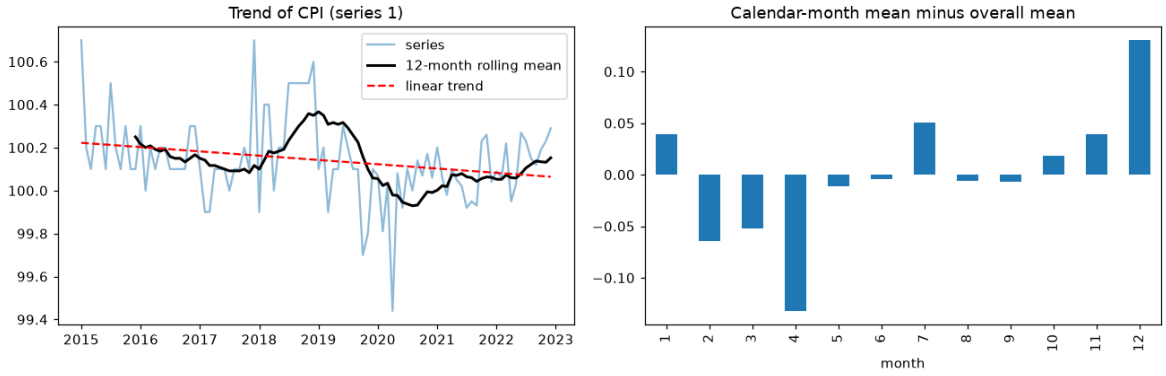
Code

```
def check_trend(s, color):
    t = np.arange(len(s))
    reg = stats.linregress(t, s.values)
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))
    ax[0].plot(s.index, s, color=color, alpha=0.5, label="series")
    ax[0].plot(s.index,  s.rolling(12).mean(),  color="black",  lw=2,  label="12-month  rolling
mean")
    ax[0].plot(s.index, reg.intercept + reg.slope * t, "r--", label="linear trend")
    ax[0].legend(); ax[0].set_title(f"Trend of {s.name}")
    month_dev = s.groupby(s.index.month).mean() - s.mean()
    month_dev.plot(kind="bar", ax=ax[1], color=color)
    ax[1].set_title("Calendar-month mean minus overall mean"); ax[1].set_xlabel("month")
    plt.tight_layout(); plt.show()
    print(f"Linear trend: slope = {reg.slope:.5f} per month, p-value = {reg.pvalue:.4f}, R^2 =
{reg.rvalue**2:.4f}")
    print("Month deviations:", month_dev.round(3).to_dict())
    print("Yearly means:"); print(s.groupby(s.index.year).mean().round(3).to_string())

check_trend(s1, "tab:blue")
```

Result

```
Linear trend: slope = -0.00166 per month, p-value = 0.0203, R^2 = 0.0560
Month deviations: {1: 0.039, 2: -0.065, 3: -0.052, 4: -0.132, 5: -0.011, 6: -0.005, 7: 0.05, 8:
-0.006, 9: -0.007, 10: 0.018, 11: 0.039, 12: 0.13}
Yearly means:
2015    100.250
2016    100.167
2017    100.117
2018    100.350
2019    100.058
2020     99.991
2021    100.062
2022    100.152
```



Comment on the result

The linear trend has a slope of -0.0017 per month (about -0.02 index points per year). Its p-value (0.020) is below 0.05, but R^2 = 0.056, so the trend explains less than 6 % of the variation and is practically negligible. The 12-month rolling mean shows medium-term swings rather than a trend: about 100.2 in 2015-2017, a peak of about 100.36 in late 2018, a trough of about 99.93 in 2020 (COVID-19) and a recovery to 100.15 in 2022. The calendar-

month profile shows a mild seasonal pattern: December (+0.13) and January (+0.04) are above the mean (Lunar New Year shopping), April (-0.13) and February-March are below it. The effect is small compared with the noise.

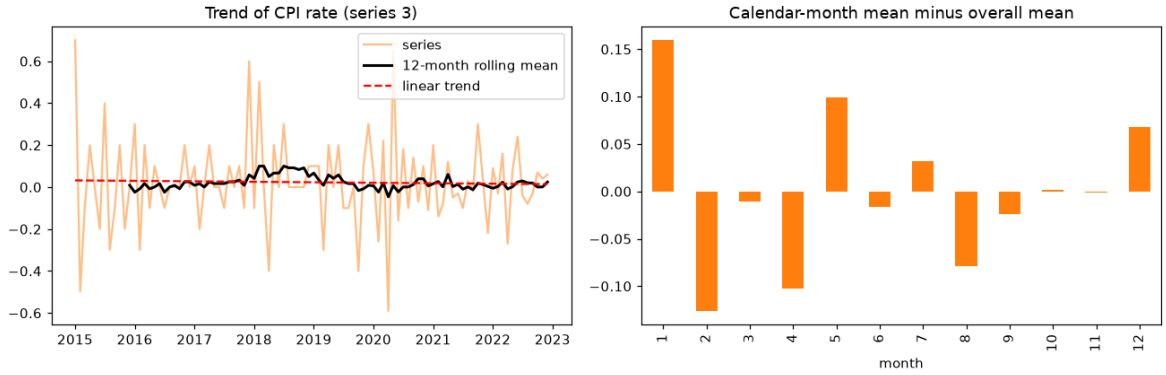
B2. CPI rate (series 3)

Code

```
check_trend(s3, "tab:orange")
```

Result

```
Linear trend: slope = -0.00019 per month, p-value = 0.8107, R^2 = 0.0006
Month deviations: {1: 0.16, 2: -0.126, 3: -0.01, 4: -0.103, 5: 0.099, 6: -0.017, 7: 0.032, 8: -
0.079, 9: -0.024, 10: 0.002, 11: -0.002, 12: 0.068}
Yearly means:
2015    0.009
2016    0.025
2017    0.058
2018    0.050
2019    0.009
2020    0.005
2021    0.003
2022    0.024
```



Comment on the result

The CPI rate has no trend: slope -0.0002 per month with p-value 0.81 and R^2 = 0.0006. The yearly means are all between 0.003 % and 0.058 %. The rolling mean stays close to zero. By calendar month, January (+0.16) and May (+0.10) have the largest increases and February (-0.13) and April (-0.10) the largest decreases, again a weak Tet-related pattern.

C. Autocorrelation function

C1. CPI (series 1)

Code

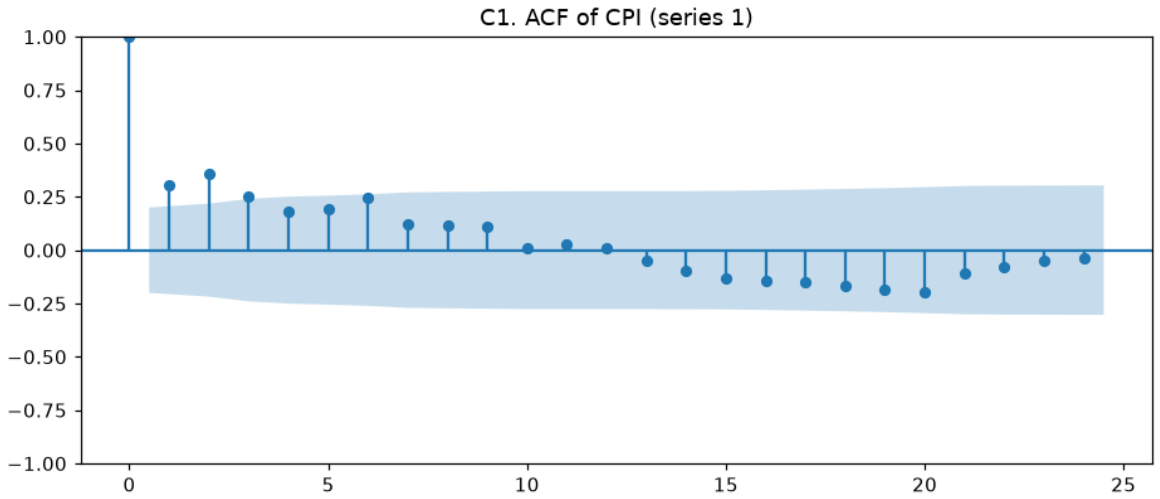
```
def significant_lags(values, confint):
```

```
    # a lag is significant when the 95% confidence band (centred on 0) does not contain the
coefficient
    half = confint[:, 1] - values
    return [k for k in range(1, len(values)) if abs(values[k]) > half[k]]

plot_acf(s1, lags=24, title="C1. ACF of CPI (series 1)"); plt.show()
a1, ci = acf(s1, nlags=24, alpha=0.05)
print("ACF lags 1-12:", np.round(a1[1:13], 3))
print("Significant ACF lags:", significant_lags(a1, ci))
```

Result

```
ACF lags 1-12: [0.306 0.355 0.253 0.18  0.193 0.247 0.123 0.115 0.108 0.009 0.029 0.01 ]
Significant ACF lags: [1, 2, 3]
```



Comment on the result

The ACF is positive and decays gradually: 0.31, 0.36 and 0.25 at lags 1-3 lie outside the 95 % confidence band; from lag 4 on the coefficients are inside the band (lag 6 = 0.25 is close to the boundary) and are about 0 at lag 12. The quick decay confirms stationarity, and the absence of a spike at lag 12 shows that yearly seasonality is weak. The ACF suggests an MA order of at most 3.

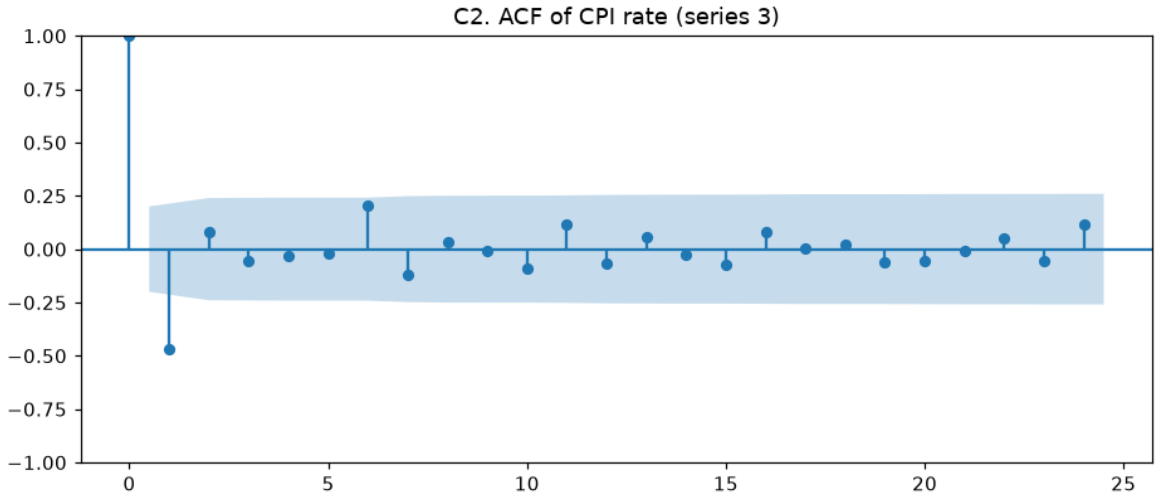
C2. CPI rate (series 3)

Code

```
plot_acf(s3, lags=24, title="C2. ACF of CPI rate (series 3)"); plt.show()
a3, ci = acf(s3, nlags=24, alpha=0.05)
print("ACF lags 1-12:", np.round(a3[1:13], 3))
print("Significant ACF lags:", significant_lags(a3, ci))
```

Result

```
ACF lags 1-12: [-0.466  0.081 -0.053 -0.034 -0.023  0.204 -0.119  0.034 -0.01  -0.089
  0.116 -0.067]
Significant ACF lags: [1]
```



Comment on the result

Only lag 1 is significant, with a strong negative value (-0.47); all other lags are inside the band. An ACF that cuts off after lag 1 is the signature of an MA(1) process. The value is close to -0.5, the theoretical lag-1 autocorrelation of a differenced white noise: the CPI rate is essentially the month-to-month change of series 1, which explains this pattern.

D. Partial autocorrelation function

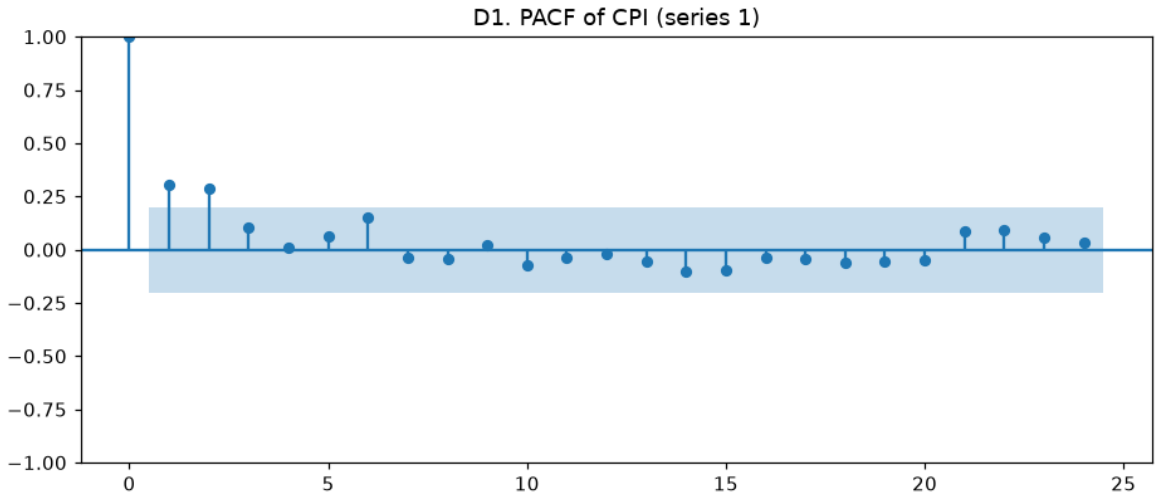
D1. CPI (series 1)

Code

```
plot_pacf(s1, lags=24, method="ywm", title="D1. PACF of CPI (series 1)"); plt.show()
pa1, ci = pacf(s1, nlags=24, method="ywm", alpha=0.05)
print("PACF lags 1-12:", np.round(pa1[1:13], 3))
print("Significant PACF lags:", significant_lags(pa1, ci))
```

Result

```
PACF lags 1-12: [ 0.306  0.289  0.105  0.007  0.065  0.153 -0.04  -0.043  0.022 -0.074
 -0.035 -0.02 ]
Significant PACF lags: [1, 2]
```



Comment on the result

The PACF has significant spikes at lags 1 (0.31) and 2 (0.29) and then cuts off (lag 3 = 0.11, all later lags inside the band). This is the pattern of an AR(2) process, so the maximum AR order is 2.

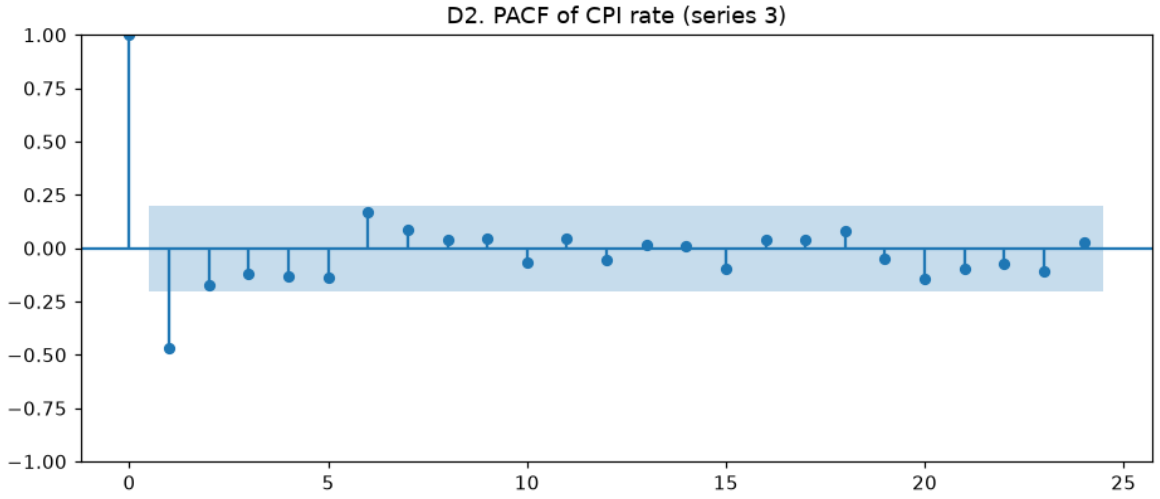
D2. CPI rate (series 3)

Code

```
plot_pacf(s3, lags=24, method="ywm", title="D2. PACF of CPI rate (series 3)"); plt.show()
pa3, ci = pacf(s3, nlags=24, method="ywm", alpha=0.05)
print("PACF lags 1-12:", np.round(pa3[1:13], 3))
print("Significant PACF lags:", significant_lags(pa3, ci))
```

Result

```
PACF lags 1-12: [-0.466 -0.174 -0.118 -0.133 -0.14   0.167  0.084  0.039  0.047 -0.07
  0.044 -0.055]
Significant PACF lags: [1]
```



Comment on the result

Only lag 1 (-0.47) is significant. Lags 2-5 are small negative values (-0.17 to -0.12) decaying slowly inside the band, which is the tail expected from an MA(1) process. The maximum AR order is 1.

E. Stationarity test

ADF test: H0 = the series has a unit root (non-stationary). KPSS test: H0 = the series is stationary. Using both tests gives a more reliable conclusion.

E1. CPI (series 1)

Code

```
def stationarity_test(s):
    adf = adfuller(s, autolag="AIC")
    print(f"ADF  statistic = {adf[0]:.4f}, p-value = {adf[1]:.6f}, lags used = {adf[2]}")
    print("     critical values:", {k: round(float(v), 3) for k, v in adf[4].items()})
    print("      ->", "stationary (reject H0)" if adf[1] < 0.05 else "non-stationary (cannot
reject H0)")
    with warnings.catch_warnings():   # KPSS warns when p-value is beyond its table (p > 0.1)
        warnings.simplefilter("ignore")
        k = kpss(s, regression="c", nlags="auto")
    print(f"KPSS statistic = {k[0]:.4f}, p-value = {k[1]:.4f}")
    print("     ->", "stationary (cannot reject H0)" if k[1] >= 0.05 else "non-stationary (reject
H0)")

stationarity_test(s1)
```

Result

```
ADF  statistic = -4.2149, p-value = 0.000622, lags used = 1
     critical values: {'1%': -3.502, '5%': -2.893, '10%': -2.583}
     -> stationary (reject H0)
KPSS statistic = 0.3602, p-value = 0.0943
     -> stationary (cannot reject H0)
```

Comment on the result

ADF statistic = -4.21 is below the 1 % critical value (-3.50), p-value = 0.0006 < 0.05, so the unit root hypothesis is rejected. KPSS statistic = 0.36 with p-value 0.094 > 0.05, so stationarity is not rejected. Both tests agree: the CPI series is stationary in levels and does not need differencing.

E2. CPI rate (series 3)

Code

```
stationarity_test(s3)
```

Result

```
ADF  statistic = -6.6319, p-value = 0.000000, lags used = 3
```

```
     critical values: {'1%': -3.504, '5%': -2.894, '10%': -2.584}
     -> stationary (reject H0)
KPSS statistic = 0.1049, p-value = 0.1000
     -> stationary (cannot reject H0)
```

Comment on the result

ADF statistic = -6.63, p-value < 0.000001: the unit root is strongly rejected. KPSS statistic = 0.10, p-value >= 0.10: stationarity is not rejected. The CPI rate is stationary.

1.3 Determine parameters p, q and d in ARIMA(p,d,q) model

Maximum value of p from the PACF

A1. CPI (series 1): the PACF is significant at lags 1 and 2 and cuts off after lag 2, so p_max = 2.

A2. CPI rate (series 3): only lag 1 of the PACF is significant, so p_max = 1.

Maximum value of q from the ACF

B1. CPI (series 1): the ACF is significant at lags 1, 2 and 3, so q_max = 3.

B2. CPI rate (series 3): the ACF cuts off after lag 1, so q_max = 1.

Value of d from the stationarity test

C1. CPI (series 1): ADF and KPSS both indicate stationarity in levels, so d = 0.

C2. CPI rate (series 3): ADF and KPSS both indicate stationarity, so d = 0.

1.4 Estimating ARIMA models and model selection

For each series all combinations p = 0..p_max, q = 0..q_max with d = 0 are estimated in a loop. A model is considered to have stable parameters when the 95 % confidence interval of every AR and MA coefficient does not contain 0 (all coefficients significant). Among the stable models the one with the lowest AIC is selected.

CPI (series 1)

A1. Estimating ARIMA(p,0,q), p = 0..2, q = 0..3

Code

```
def quiet_fit(data, order, seasonal_order=(0, 0, 0, 0)):
    # statsmodels re-enables its own warnings inside fit(), so silence them here
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        return ARIMA(data, order=order, seasonal_order=seasonal_order).fit()

def arima_grid(s, p_max, d, q_max):
```

```
    rows = []
    for p in range(p_max + 1):
        for q in range(q_max + 1):
            res = quiet_fit(s, (p, d, q))
            ci = res.conf_int()
            arma = [n for n in res.params.index if n.startswith(("ar.", "ma."))]
            stable = all(not (ci.loc[n, 0] <= 0 <= ci.loc[n, 1]) for n in arma)
            rows.append({"order": (p, d, q), "AIC": round(res.aic, 3), "BIC": round(res.bic, 3),
"stable": stable})
    table = pd.DataFrame(rows).sort_values("AIC").reset_index(drop=True)
    best = table[table.stable].iloc[0]["order"]
    return table, best

P1, D1, Q1 = 2, 0, 3   # maxima from step 1.3
table1, best1 = arima_grid(s1, P1, D1, Q1)
print(table1.to_string())
```

Result

```
         order     AIC     BIC  stable
0   (1, 0, 1) -52.988 -42.730    True
1   (2, 0, 0) -52.839 -42.581    True
2   (2, 0, 1) -51.933 -39.112   False
3   (1, 0, 2) -51.681 -38.859   False
4   (1, 0, 3) -50.525 -35.139   False
5   (2, 0, 2) -50.039 -34.653   False
6   (0, 0, 3) -49.778 -36.957   False
7   (0, 0, 2) -48.961 -38.704   False
8   (2, 0, 3) -48.529 -30.579   False
9   (1, 0, 0) -45.908 -38.215    True
10  (0, 0, 1) -41.859 -34.166    True
11  (0, 0, 0) -37.576 -32.447    True
```

A2. Selecting the best ARIMA model

Code

```
print("Best stable model for CPI (series 1): ARIMA", best1)
best_fit1 = quiet_fit(s1, best1)
print(best_fit1.summary().tables[1])
print(f"AIC = {best_fit1.aic:.3f}")
```

Result

```
Best stable model for CPI (series 1): ARIMA (1, 0, 1)
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const        100.1593      0.045   2222.718      0.000     100.071     100.248
ar.L1          0.8742      0.088      9.912      0.000       0.701       1.047
ma.L1         -0.6316      0.143     -4.426      0.000      -0.911      -0.352
sigma2         0.0309      0.003      9.133      0.000       0.024       0.038
==============================================================================
AIC = -52.988
```

Comment on the result

Twelve models were estimated. ARIMA(1,0,1) has the lowest AIC (-52.99) and all its coefficients are significant: ar.L1 = 0.874 (CI 0.70 to 1.05) and ma.L1 =

-0.632 (CI -0.91 to -0.35). ARIMA(2,0,0) is almost as good (AIC -52.84) and also stable; larger models such as ARIMA(2,0,1) or ARIMA(1,0,2) have higher AIC and contain insignificant coefficients (over-fitting). The best model is therefore ARIMA(pO,dO,qO) = ARIMA(1,0,1): CPI(t) = 100.159 + 0.874 (CPI(t-1) - 100.159) + e(t) - 0.632 e(t-1). The AR coefficient close to 1 shows that deviations from the mean decay slowly (persistence).

CPI rate (series 3)

B1. Estimating ARIMA(p,0,q), p = 0..1, q = 0..1

Code

```
P3, D3, Q3 = 1, 0, 1   # maxima from step 1.3
table3, best3 = arima_grid(s3, P3, D3, Q3)
print(table3.to_string())
```

Result

```
       order     AIC     BIC  stable
0  (0, 0, 1) -48.960 -41.267    True
1  (1, 0, 1) -47.754 -37.497   False
2  (1, 0, 0) -45.260 -37.567    True
3  (0, 0, 0) -20.767 -15.639    True
```

B2. Selecting the best ARIMA model

Code

```
print("Best stable model for CPI rate (series 3): ARIMA", best3)
best_fit3 = quiet_fit(s3, best3)
print(best_fit3.summary().tables[1])
print(f"AIC = {best_fit3.aic:.3f}")
```

Result

```
Best stable model for CPI rate (series 3): ARIMA (0, 0, 1)
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0205      0.009      2.365      0.018       0.004       0.037
ma.L1         -0.5688      0.062     -9.230      0.000      -0.690      -0.448
sigma2         0.0329      0.003      9.858      0.000       0.026       0.039
==============================================================================
AIC = -48.960
```

Comment on the result

ARIMA(0,0,1) has the lowest AIC (-48.96) and is stable: ma.L1 = -0.569 (CI -0.69 to -0.45) and the constant 0.0205 (CI 0.004 to 0.037) are significant. ARIMA(1,0,1) has a higher AIC and an insignificant coefficient, ARIMA(1,0,0) is

stable but has a higher AIC. The best model is ARIMA(pO,dO,qO) = ARIMA(0,0,1): rate(t) = 0.0205 + e(t) - 0.569 e(t-1), in agreement with the ACF/PACF analysis.

1.5 Mean values forecasting for ARIMA model

CPI (series 1) - ARIMA(1,0,1), 12 months ahead

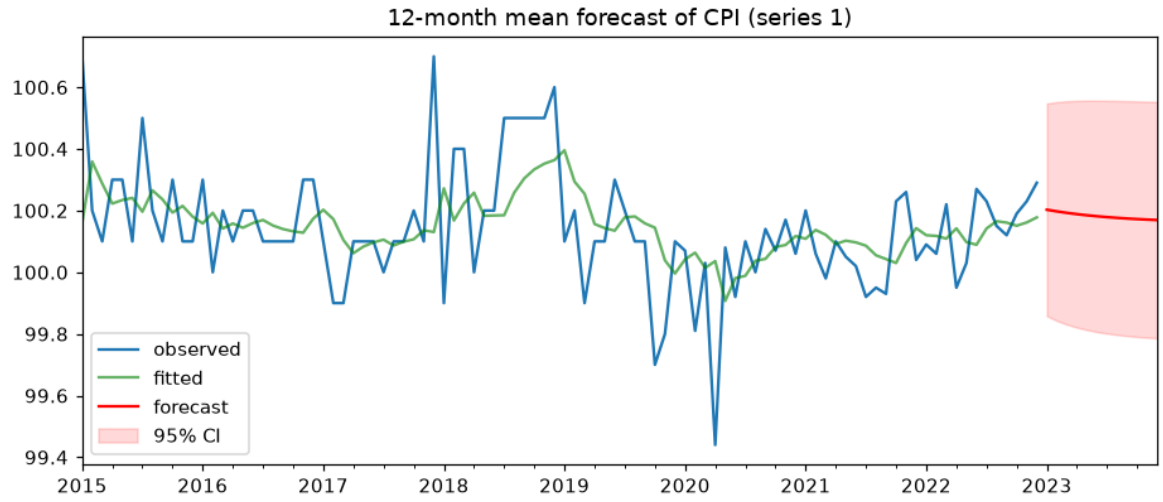
Code

```
def forecast_mean(fit, s, steps=12, color="tab:blue"):
    fc = fit.get_forecast(steps=steps)
    mean, ci = fc.predicted_mean, fc.conf_int(alpha=0.05)
    out = pd.DataFrame({"forecast": mean, "lower 95%": ci.iloc[:, 0], "upper 95%": ci.iloc[:,
1]}).round(3)
    out.index = out.index.strftime("%Y-%m")
    print(out.to_string())
    ax = s.plot(color=color, label="observed")
    fit.fittedvalues.plot(ax=ax, color="green", alpha=0.6, label="fitted")
    mean.plot(ax=ax, color="red", label="forecast")
    ax.fill_between(ci.index, ci.iloc[:, 0], ci.iloc[:, 1], color="red", alpha=0.15, label="95%
CI")
    ax.legend(); ax.set_title(f"12-month mean forecast of {s.name}"); plt.show()

forecast_mean(best_fit1, s1)
```

Result

```
         forecast  lower 95%  upper 95%
2023-01   100.203     99.858    100.547
2023-02   100.197     99.843    100.552
2023-03   100.192     99.830    100.554
2023-04   100.188     99.821    100.556
2023-05   100.185     99.813    100.556
2023-06   100.181     99.806    100.556
2023-07   100.179     99.801    100.556
2023-08   100.176     99.797    100.555
2023-09   100.174     99.793    100.555
2023-10   100.172     99.791    100.554
2023-11   100.171     99.788    100.553
2023-12   100.169     99.786    100.552
```



Comment on the result

The forecast for January 2023 is 100.20 and it decreases smoothly to 100.17 in December 2023, converging to the long-run mean 100.16 at the speed given by the AR coefficient 0.874. The 95 % confidence interval is about 99.79-100.55 and widens only slightly with the horizon because the series is stationary. The fitted values follow the general level of the series but smooth out the isolated spikes (December 2017, April 2020).

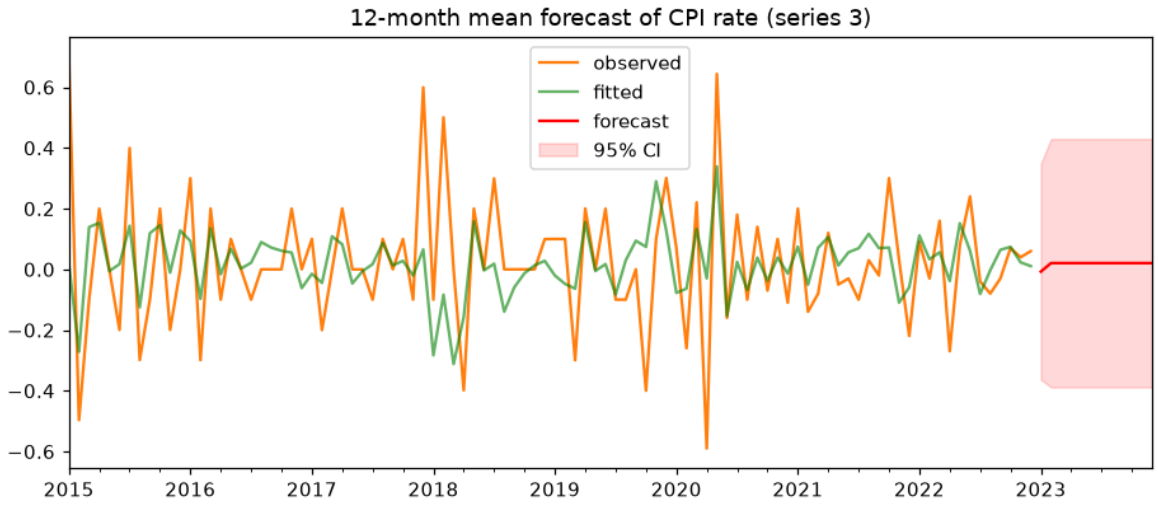
CPI rate (series 3) - ARIMA(0,0,1), 12 months ahead

Code

```
forecast_mean(best_fit3, s3, color="tab:orange")
```

Result

	forecast	lower 95%	upper 95%
2023-01	-0.007	-0.363	0.348
2023-02	0.020	-0.388	0.429
2023-03	0.020	-0.388	0.429
2023-04	0.020	-0.388	0.429
2023-05	0.020	-0.388	0.429
2023-06	0.020	-0.388	0.429
2023-07	0.020	-0.388	0.429
2023-08	0.020	-0.388	0.429
2023-09	0.020	-0.388	0.429
2023-10	0.020	-0.388	0.429
2023-11	0.020	-0.388	0.429
2023-12	0.020	-0.388	0.429



Comment on the result

An MA(1) model only remembers one past shock, so the forecast for January 2023 (-0.007 %) uses the last residual and from February 2023 it equals the

mean 0.020 % per month. The 95 % interval (-0.39 % to +0.43 %) is wide compared with the mean: month-to-month changes of garment prices are hard to predict beyond one step.

1.6 Machine learning in ARIMA model forecasting

The series is split chronologically: the first 80 % (76 months) is the training part and the last 20 % (20 months) the test part. Walk-forward validation is used: at each test month the model is re-estimated on all data available up to that month, a one-step-ahead forecast is made, then the true value is added to the history. The accuracy is measured by RMSE on the test part. The forecast is also repeated for all (p,d,q) combinations of step 1.4 to find the model with the lowest RMSE.

CPI (series 1)

A1. Train / test split

Code

```
def split(s, ratio=0.8):
    n = int(len(s) * ratio)
    return s.iloc[:n], s.iloc[n:]

def walk_forward(s, order, seasonal_order=(0, 0, 0, 0)):
    train, test = split(s)
    history, preds = list(train), []
    for actual in test:
        fit = quiet_fit(history, order, seasonal_order)
        preds.append(fit.forecast()[0])
        history.append(actual)
    preds = pd.Series(preds, index=test.index)
    rmse = np.sqrt(mean_squared_error(test, preds))
    return preds, rmse

def plot_test(s, preds, title):
    train, test = split(s)
    ax = train.plot(label="train", color="grey")
    test.plot(ax=ax, label="test (actual)", color="tab:blue", marker="o", ms=3)
    preds.plot(ax=ax, label="prediction", color="red", marker="x", ms=4)
    ax.legend(); ax.set_title(title); plt.show()

train1, test1 = split(s1)
print(f"Train: {train1.index[0]:%Y-%m} .. {train1.index[-1]:%Y-%m}  ({len(train1)} obs)")
print(f"Test : {test1.index[0]:%Y-%m} .. {test1.index[-1]:%Y-%m}  ({len(test1)} obs)")
```

Result

```
Train: 2015-01 .. 2021-04  (76 obs)
Test : 2021-05 .. 2022-12  (20 obs)
```

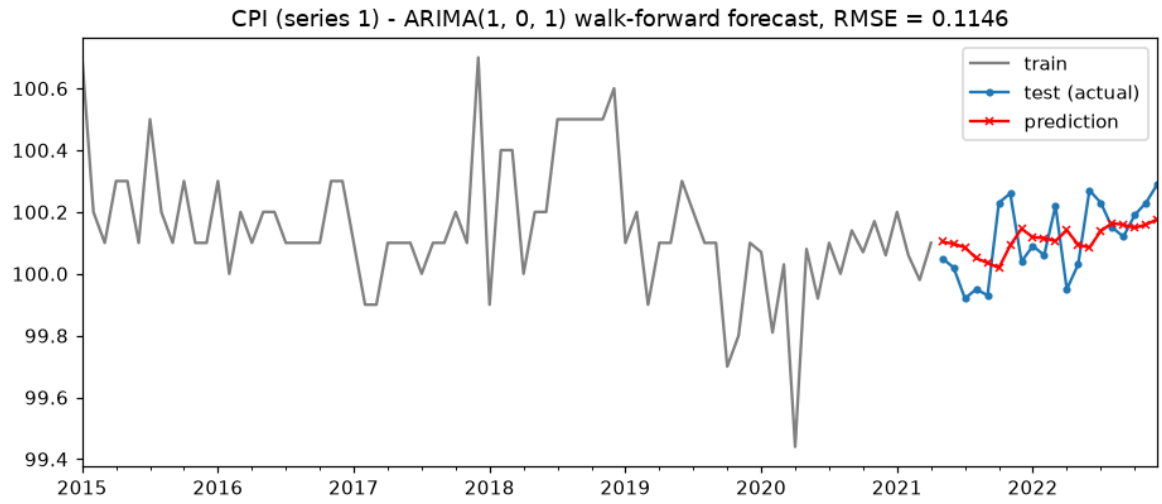
A2. Forecasting with ARIMA(1,0,1) and with all combinations

Code

```
preds1, rmse1 = walk_forward(s1, best1)
print(f"ARIMA{best1}: test RMSE = {rmse1:.4f}")
plot_test(s1, preds1, f"CPI (series 1) - ARIMA{best1} walk-forward forecast, RMSE = {rmse1:.4f}")
rmse_grid1 = pd.DataFrame([{"order": str(o), "RMSE": round(walk_forward(s1, o)[1], 4)} for o in
table1.order])
print(rmse_grid1.sort_values("RMSE").to_string(index=False))
```

Result

```
ARIMA(1, 0, 1): test RMSE = 0.1146
    order   RMSE
(1, 0, 1) 0.1146
(1, 0, 0) 0.1149
(0, 0, 1) 0.1179
(2, 0, 2) 0.1211
(2, 0, 1) 0.1213
(2, 0, 0) 0.1226
(1, 0, 2) 0.1230
(1, 0, 3) 0.1251
(2, 0, 3) 0.1257
(0, 0, 0) 0.1263
(0, 0, 2) 0.1325
(0, 0, 3) 0.1348
```



Comment on the result

ARIMA(1,0,1) reaches a test RMSE of 0.1146 index points, the lowest among all 12 combinations, so the RMSE criterion confirms the AIC choice. ARIMA(1,0,0) is almost equal (0.1149), while the constant-mean model ARIMA(0,0,0) gives 0.1263, i.e. the ARIMA dynamics reduce the error by about 9 %. The predictions follow the level of the test data (the dip in mid-2021 and the rise in late 2022) but cannot anticipate individual monthly jumps.

CPI rate (series 3)

B1. Train / test split

Code

```
train3, test3 = split(s3)
print(f"Train: {train3.index[0]:%Y-%m} .. {train3.index[-1]:%Y-%m}  ({len(train3)} obs)")
print(f"Test : {test3.index[0]:%Y-%m} .. {test3.index[-1]:%Y-%m}  ({len(test3)} obs)")
```

Result

```
Train: 2015-01 .. 2021-04  (76 obs)
Test : 2021-05 .. 2022-12  (20 obs)
```

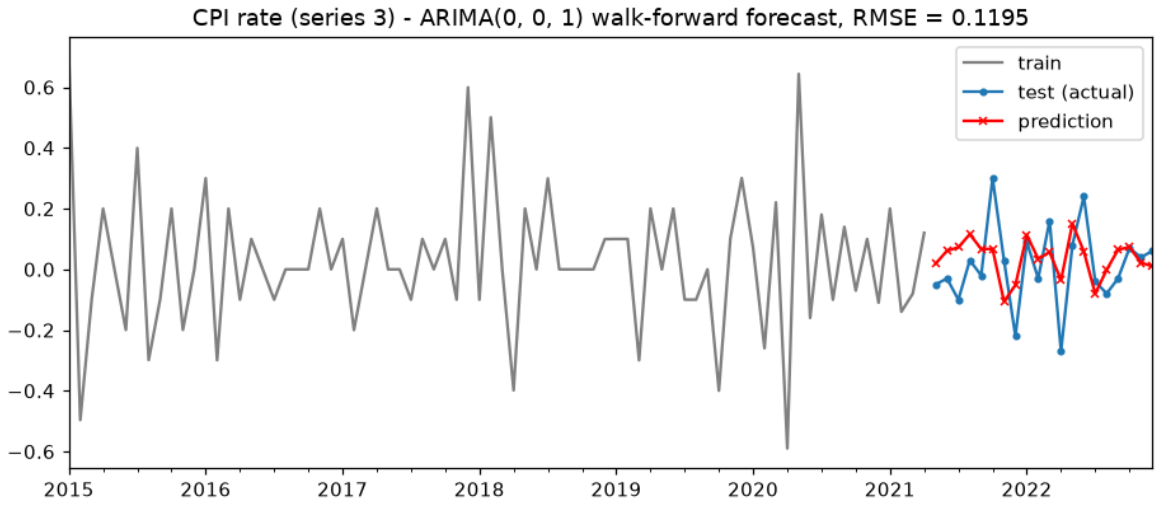
B2. Forecasting with ARIMA(0,0,1) and with all combinations

Code

```
preds3, rmse3 = walk_forward(s3, best3)
print(f"ARIMA{best3}: test RMSE = {rmse3:.4f}")
plot_test(s3, preds3, f"CPI rate (series 3) - ARIMA{best3} walk-forward forecast, RMSE = {rmse3:.4f}")
rmse_grid3 = pd.DataFrame([{"order": str(o), "RMSE": round(walk_forward(s3, o)[1], 4)} for o in table3.order])
print(rmse_grid3.sort_values("RMSE").to_string(index=False))
```

Result

```
ARIMA(0, 0, 1): test RMSE = 0.1195
    order   RMSE
(0, 0, 1) 0.1195
(1, 0, 1) 0.1239
(0, 0, 0) 0.1320
(1, 0, 0) 0.1337
```



Comment on the result

ARIMA(0,0,1) gives the lowest test RMSE (0.1195 percentage points) of the 4 combinations, again agreeing with the AIC selection; the constant-mean model gives 0.1320 (about 9 % worse). The MA(1) term captures the tendency of a large monthly change to be partly reversed in the next month.

1.7 Machine learning in SARIMA model forecasting

SARIMA(pO,dO,qO)(P,D,Q,m) adds seasonal terms with m = 12 and D = 0 to the optimal ARIMA orders. The same 80/20 split and walk-forward procedure are used. P and Q are varied over 0, 1, 2; the combination (0,0,0,12) is the plain ARIMA model and is kept in the table only as a baseline.

CPI (series 1) - SARIMA(1,0,1)(P,0,Q,12)

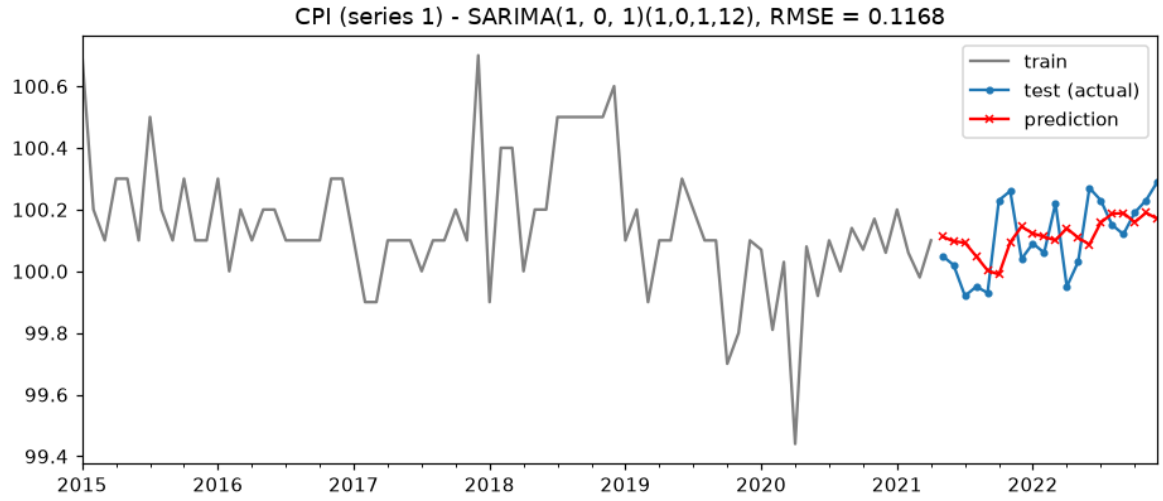
A1. Forecast with (P,D,Q,m) = (1,0,1,12)

Code

```
sp1, srmse1 = walk_forward(s1, best1, (1, 0, 1, 12))
print(f"SARIMA{best1}(1,0,1,12): test RMSE = {srmse1:.4f}")
plot_test(s1, sp1, f"CPI (series 1) - SARIMA{best1}(1,0,1,12), RMSE = {srmse1:.4f}")
```

Result

```
SARIMA(1, 0, 1)(1,0,1,12): test RMSE = 0.1168
```



A2. Loop over P, Q = 0..2

Code

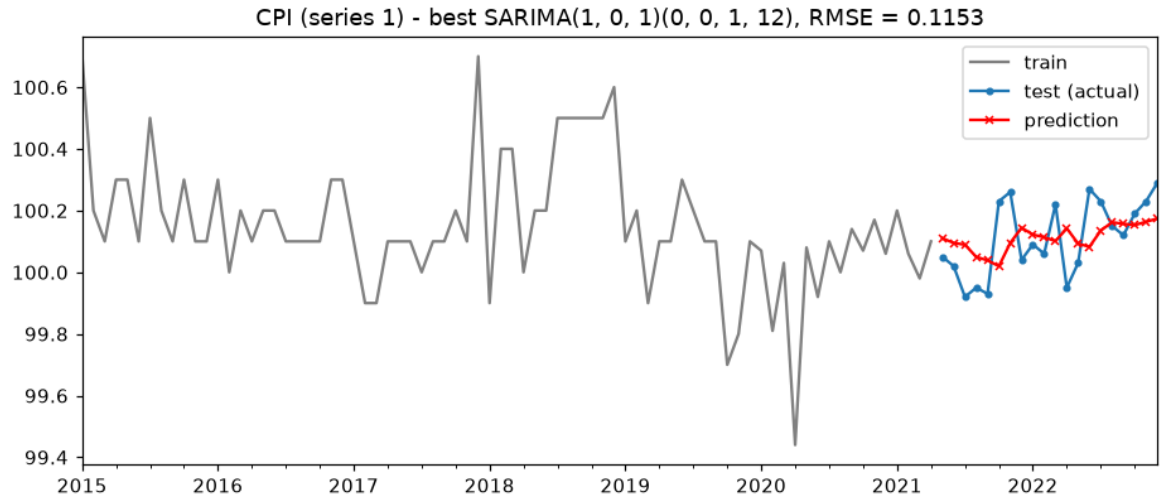
```
def sarima_grid(s, order, PQ=range(3)):
    rows = []
    for P in PQ:
        for Q in PQ:
            rows.append({"seasonal (P,D,Q,m)": (P, 0, Q, 12), "RMSE": round(walk_forward(s,
order, (P, 0, Q, 12))[1], 4)})
    table = pd.DataFrame(rows).sort_values("RMSE").reset_index(drop=True)
    # (0,0,0,12) is plain ARIMA: keep it as the baseline but pick the best truly seasonal model
    best = table[table["seasonal (P,D,Q,m)"] != (0, 0, 0, 12)].iloc[0]["seasonal (P,D,Q,m)"]
    return table, best

sg1, bs1 = sarima_grid(s1, best1)
print(sg1.to_string())
bp1, brmse1 = walk_forward(s1, best1, bs1)
print(f"Best: SARIMA{best1}{bs1}, RMSE = {brmse1:.4f}  (plain ARIMA{best1}: {rmse1:.4f})")
```

```
plot_test(s1, bp1, f"CPI (series 1) - best SARIMA{best1}{bs1}, RMSE = {brmse1:.4f}")
```

Result

```
  seasonal (P,D,Q,m)    RMSE
0      (0, 0, 0, 12)  0.1146
1      (0, 0, 1, 12)  0.1153
2      (1, 0, 0, 12)  0.1154
3      (1, 0, 1, 12)  0.1168
4      (2, 0, 1, 12)  0.1196
5      (2, 0, 0, 12)  0.1202
6      (0, 0, 2, 12)  0.1216
7      (1, 0, 2, 12)  0.1249
8      (2, 0, 2, 12)  0.1261
Best: SARIMA(1, 0, 1)(0, 0, 1, 12), RMSE = 0.1153  (plain ARIMA(1, 0, 1): 0.1146)
```



Comment on the result

SARIMA(1,0,1)(1,0,1,12) gives RMSE 0.1168, worse than ARIMA(1,0,1) (0.1146). In the loop the best seasonal model is SARIMA(1,0,1)(0,0,1,12) with RMSE 0.1153, still slightly worse than the non-seasonal baseline, and models with more seasonal terms are worse (up to 0.126). This agrees with the ACF (autocorrelation at lag 12 is about 0.01) and the small calendar-month effects: the yearly seasonality of garment prices is too weak to be exploited, and the extra parameters only add estimation noise.

CPI rate (series 3) - SARIMA(0,0,1)(P,0,Q,12)

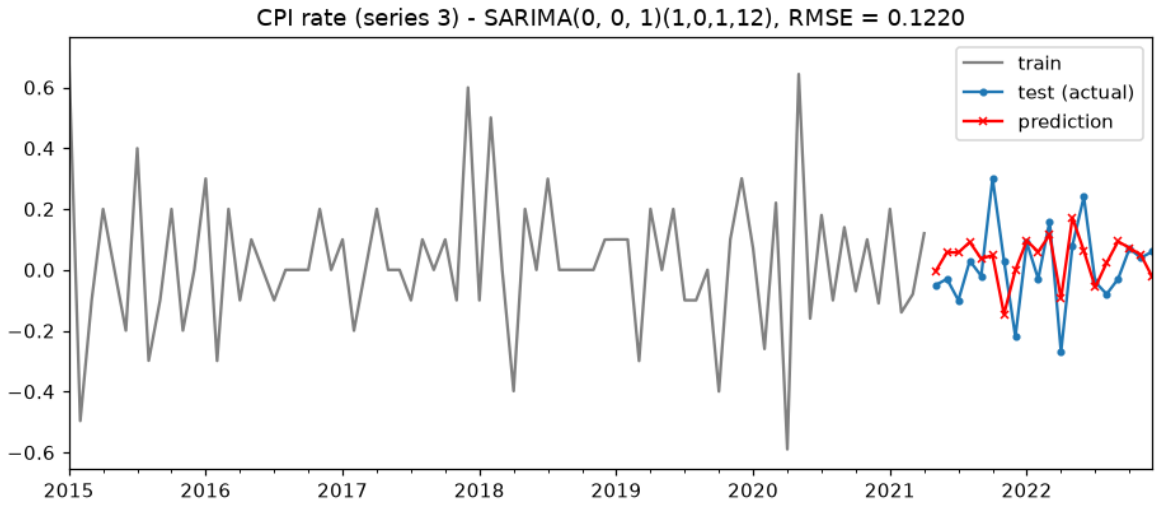
B1. Forecast with (P,D,Q,m) = (1,0,1,12)

Code

```
sp3, srmse3 = walk_forward(s3, best3, (1, 0, 1, 12))
print(f"SARIMA{best3}(1,0,1,12): test RMSE = {srmse3:.4f}")
plot_test(s3, sp3, f"CPI rate (series 3) - SARIMA{best3}(1,0,1,12), RMSE = {srmse3:.4f}")
```

Result

```
SARIMA(0, 0, 1)(1,0,1,12): test RMSE = 0.1220
```



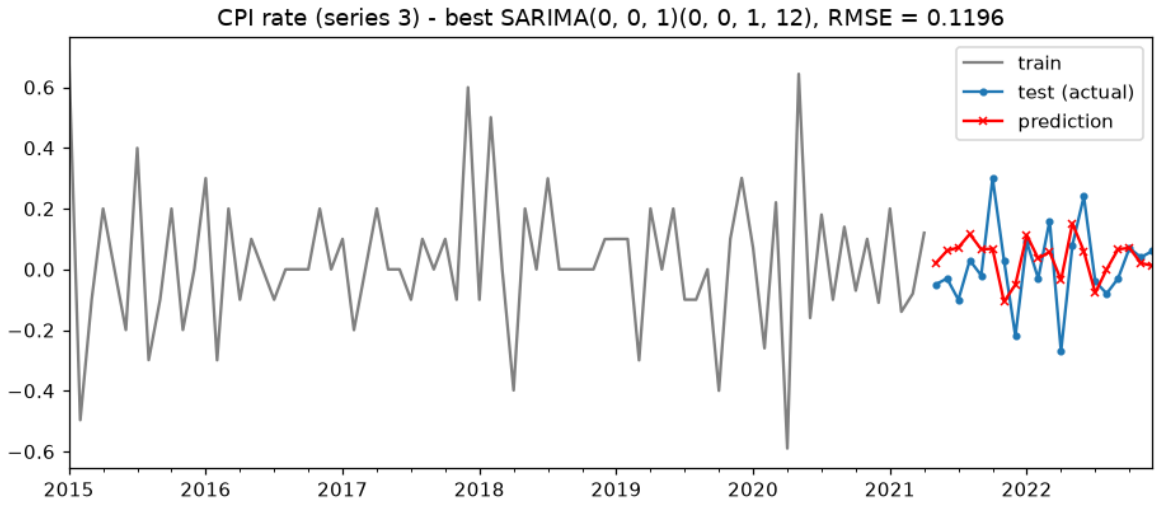
B2. Loop over P, Q = 0..2

Code

```
sg3, bs3 = sarima_grid(s3, best3)
print(sg3.to_string())
bp3, brmse3 = walk_forward(s3, best3, bs3)
print(f"Best: SARIMA{best3}{bs3}, RMSE = {brmse3:.4f}  (plain ARIMA{best3}: {rmse3:.4f})")
plot_test(s3, bp3, f"CPI rate (series 3) - best SARIMA{best3}{bs3}, RMSE = {brmse3:.4f}")
```

Result

```
  seasonal (P,D,Q,m)    RMSE
0      (0, 0, 0, 12)  0.1195
1      (0, 0, 1, 12)  0.1196
2      (1, 0, 0, 12)  0.1196
3      (1, 0, 1, 12)  0.1220
4      (2, 0, 0, 12)  0.1258
5      (0, 0, 2, 12)  0.1261
6      (2, 0, 1, 12)  0.1282
7      (2, 0, 2, 12)  0.1287
8      (1, 0, 2, 12)  0.1309
Best: SARIMA(0, 0, 1)(0, 0, 1, 12), RMSE = 0.1196  (plain ARIMA(0, 0, 1): 0.1195)
```



Comment on the result

SARIMA(0,0,1)(1,0,1,12) gives RMSE 0.1220 versus 0.1195 for ARIMA(0,0,1). The best seasonal model is SARIMA(0,0,1)(0,0,1,12) with RMSE 0.1196, practically identical to the plain ARIMA. Seasonal terms do not improve the forecast of the CPI rate either.

Part 1 conclusion

The Hanoi garment, footwear and hat CPI (2015-2022) fluctuates in a narrow range around 100.14 with no meaningful trend, weak seasonality (higher around Tet, lower in April) and shocks in 2018 and 2020 (COVID-19). ADF and KPSS tests show that both the CPI and the CPI rate are stationary, so d = 0. The ACF/PACF identify an AR(2)/ARMA structure for the CPI and an MA(1) structure for the CPI rate. The best models by AIC with stable (significant) parameters are ARIMA(1,0,1) for the CPI and ARIMA(0,0,1) for the CPI rate; the walk-forward train/test evaluation selects exactly the same models by RMSE (0.1146 and 0.1195), about 9 % better than a constant-mean forecast. Adding seasonal SARIMA terms with m = 12 does not reduce the RMSE, so the non-seasonal ARIMA models are recommended. The forecast for 2023 is a CPI of about 100.2 and a monthly CPI rate of about 0.02 %, with wide uncertainty.

Part 2. Detecting anomalies and change points in time series of Asian economic data

Summary: Five annual indicators of Indonesia (1990-2021) are examined. The Z-score method flags only the 1998 inflation (58 %). Isolation Forest flags extreme years in every series. PELT finds structural changes around 2010 for GDP, green GDP, FDI and inflation, and in 1996 and 2006 for military spending.

2.1 Introduction

Methodology of anomaly and change point detection

An anomaly (outlier) is an observation that deviates strongly from the rest of the series; a change point is a time where the statistical properties of the series (mean, variance, distribution) change and stay changed. Three methods are used: (1) the Z-score method standardises each value, z = (x - mean) / std, and flags |z| > 3; it assumes an approximately normal, stationary series. (2) Isolation Forest (Liu, Ting and Zhou, 2008) builds random trees that split the data at random values; anomalies are isolated in fewer splits and get a lower score. The parameter contamination gives the expected share of anomalies (here 0.1, about 3 of 32 years). (3) Change point detection with the PELT algorithm (Killick et al., 2012) of the ruptures library finds the segmentation that minimises a cost (here the RBF kernel cost on the standardised series) plus a penalty (pen = 3) for each change point; each segment has at least 3 years.

Economic indicators

+ gdp - Gross Domestic Product: total value of final goods and services produced in a year (current USD).
+ ggdp - Green GDP: GDP adjusted for the depletion of natural resources and environmental damage (USD).
+ inf - inflation rate: annual percentage change of consumer prices (%).
+ fdi - foreign direct investment, net inflows (USD); negative values mean disinvestment by foreign investors.
+ mil - government military expenditure as a percentage of GDP (%).

Data array "Data02_05_Indonesia_C.csv"

Annual data of Indonesia from 1990 to 2021 (32 years). ggdp is missing for 2020-2021 and mil for 1995; missing years are skipped (not imputed) for each series.

+ Column 1: year
+ Column 2: country (Indonesia)
+ Column 3: ggdp = Green Gross Domestic Product
+ Column 4: mil = Government spending on military (% of GDP)
+ Column 5: inf = Inflation rate (%)
+ Column 6: fdi = Foreign direct investment (USD)
+ Column 7: gdp = Gross Domestic Product (USD)

Code

```
eco = pd.read_csv("GrProjEconData/Data02_05_Indonesia_C.csv", index_col="year")
print(eco.drop(columns="country").describe().T.round(3))
print("Missing values:", eco.isna().sum()[eco.isna().sum() > 0].to_dict())
COLS = ["ggdp", "mil", "inf", "fdi", "gdp"]
NAMES = {"ggdp": "Green GDP (USD)", "mil": "Military spending (% of GDP)", "inf": "Inflation
(%)",
         "fdi": "FDI net inflows (USD)", "gdp": "GDP (USD)"}

def get(col):
    return eco[col].dropna()   # missing years are skipped, not imputed
```

Result

```
      count          mean           std           min           25%  \
ggdp   30.0  4.467000e+11  3.461709e+11  8.600000e+10  1.550000e+11
mil    31.0  1.192000e+00  6.790000e-01  5.710000e-01  7.060000e-01
inf    32.0  8.781000e+00  9.832000e+00  1.560000e+00  4.360000e+00
fdi    32.0  8.887605e+09  9.445324e+09 -4.550355e+09  1.703250e+09
gdp    32.0  5.095351e+11  3.804358e+11  9.544555e+10  1.739242e+11

               50%           75%           max
ggdp  2.500000e+11  8.475000e+11  1.100000e+12
mil   9.190000e-01  1.340000e+00  2.883000e+00
inf   6.410000e+00  9.483000e+00  5.845100e+01
fdi   4.895785e+09  1.932609e+10  2.512073e+10
gdp   3.255000e+11  8.980000e+11  1.190000e+12
Missing values: {'ggdp': 2, 'mil': 1}
```

2.2 Detecting anomalies

Z-Score Method (threshold = 3)

A1. Series 1: ggdp

Code

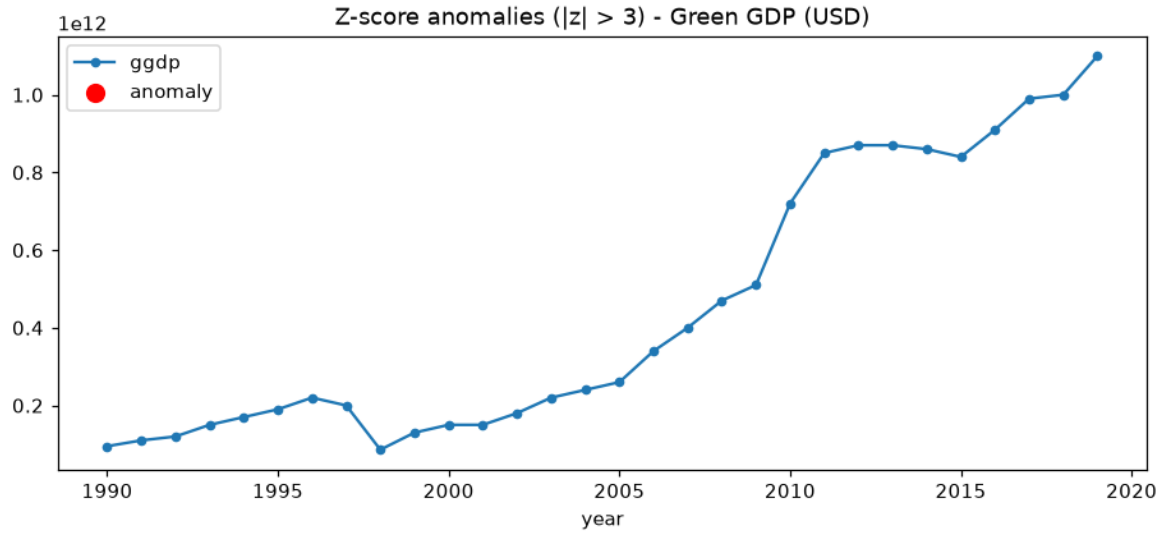
```
def zscore_anomalies(col, threshold=3):
    s = get(col)
```

```
    z = (s - s.mean()) / s.std()
    out = s[z.abs() > threshold]
    ax = s.plot(marker="o", ms=4, title=f"Z-score anomalies (|z| > {threshold}) - {NAMES[col]}")
    ax.scatter(out.index, out, color="red", s=80, zorder=3, label="anomaly")
    ax.legend(); plt.show()
    print(pd.DataFrame({"value":                                s,                                "z":
z.round(3)}).loc[z.abs().sort_values(ascending=False).index[:3]])
    print("Anomalies:", {int(y): round(float(v), 3) for y, v in out.items()} or "none")

zscore_anomalies("ggdp")
```

Result

```
              value      z
year
2019  1.100000e+12  1.887
2018  1.000000e+12  1.598
2017  9.900000e+11  1.569
Anomalies: none
```



Comment on the result

No anomaly: the largest |z| is 1.89 (2019). Green GDP grows from 95 to 1,100 billion USD, so its mean and standard deviation are dominated by the trend and no single year is far from them. The 1998 crisis drop (200 to 86 billion USD) is not detected because the value is low but not extreme compared with the whole sample.

A2. Series 2: mil

Code

```
zscore_anomalies("mil")
```

Result

```
         value      z
year
1994  2.882670  2.489
1990  2.543151  1.989
1991  2.531463  1.972
Anomalies: none
```



Comment on the result

No anomaly. The highest value, 2.88 % in 1994, has z = 2.49 < 3. Military spending declines from about 2.5 % of GDP in the early 1990s to 0.6-0.9 % after 2006, so early years are the farthest from the mean but not beyond the threshold.

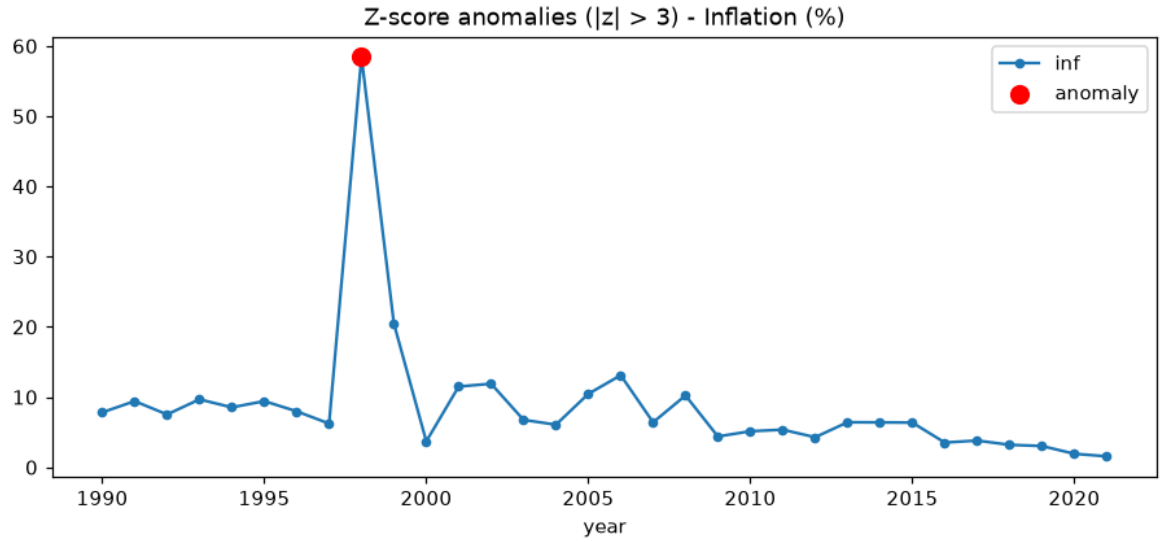
A3. Series 3: inf

Code

```
zscore_anomalies("inf")
```

Result

```
           value      z
year
1998  58.451044  5.052
1999  20.477831  1.190
2021   1.560130 -0.734
Anomalies: {1998: 58.451}
```



Comment on the result

One anomaly: 1998 with inflation of 58.45 % (z = 5.05), caused by the Asian financial crisis and the collapse of the rupiah. The 1999 value (20.5 %) is also high but has z = 1.19 only, because the 1998 value itself inflates the standard deviation (masking effect).

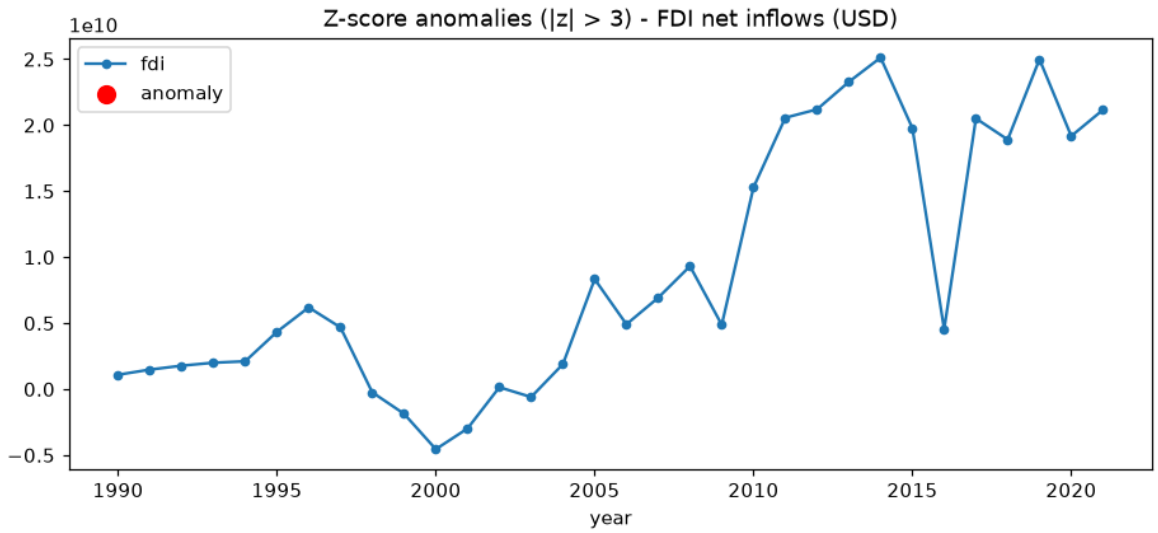
A4. Series 4: fdi

Code

```
zscore_anomalies("fdi")
```

Result

```
              value      z
year
2014  2.512073e+10  1.719
2019  2.499355e+10  1.705
2013  2.328174e+10  1.524
Anomalies: none
```



Comment on the result

No anomaly: the maximum |z| is 1.72 (2014). The negative FDI of 1998-2001 (net outflows up to -4.55 billion USD in 2000) is economically abnormal but not extreme in z-score terms because of the strong upward trend after 2010.

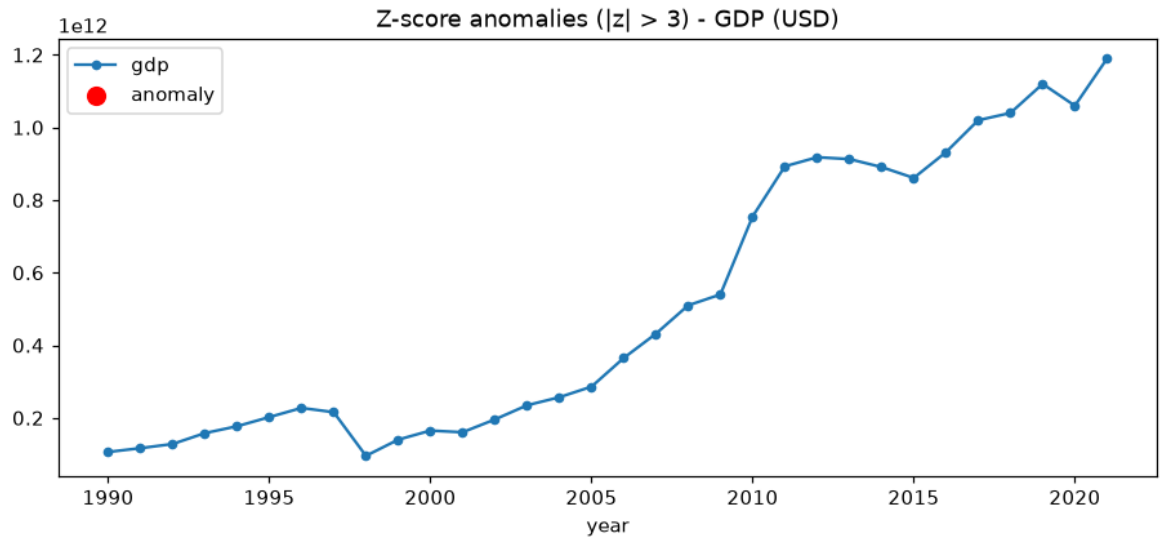
A5. Series 5: gdp

Code

```
zscore_anomalies("gdp")
```

Result

```
               value      z
year
2021  1190000000000  1.789
2019  1120000000000  1.605
2020  1060000000000  1.447
Anomalies: none
```



Comment on the result

No anomaly: the maximum |z| is 1.79 (2021). As for green GDP, the Z-score method is not suitable for a strongly trending series; the crisis of 1998 (GDP falling from 216 to 95 billion USD) is not detected.

Isolation Forest Method

A1. Series 1: ggdp

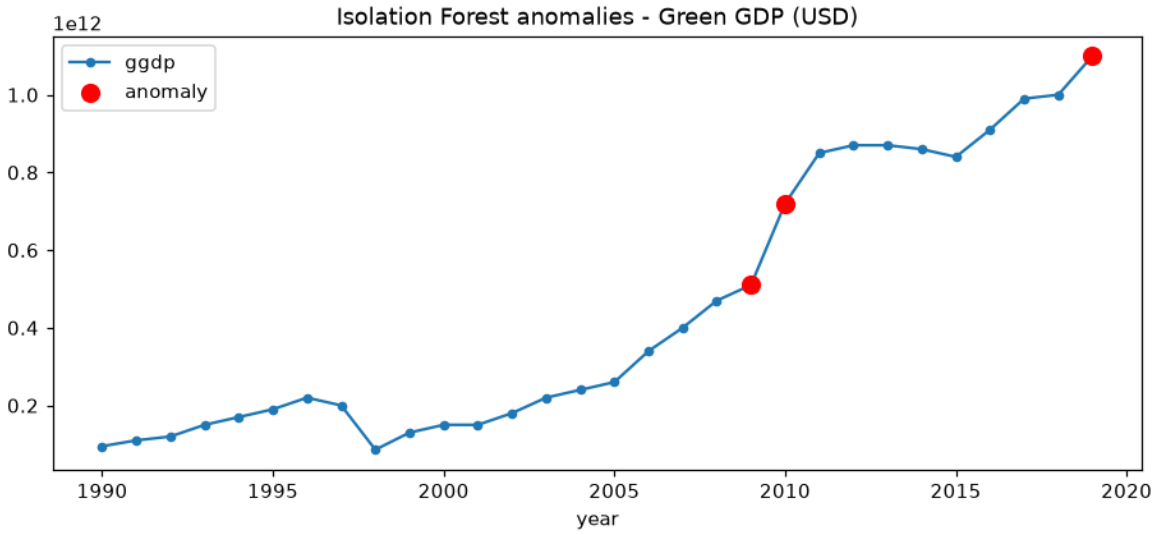
Code

```
def iforest_anomalies(col, contamination=0.1):
    s = get(col)
    model = IsolationForest(n_estimators=200, contamination=contamination, random_state=42)
    X = s.values.reshape(-1, 1)
    label = model.fit_predict(X)
    score = model.decision_function(X)
    out = s[label == -1]
    ax = s.plot(marker="o", ms=4, title=f"Isolation Forest anomalies - {NAMES[col]}")
    ax.scatter(out.index, out, color="red", s=80, zorder=3, label="anomaly")
    ax.legend(); plt.show()
    print(pd.DataFrame({"value": out, "score": score[label == -1].round(4)}))

iforest_anomalies("ggdp")
```

Result

```
             value   score
year
2009  5.100000e+11 -0.0099
2010  7.200000e+11 -0.0361
2019  1.100000e+12 -0.1322
```



Comment on the result

Isolation Forest flags 2009, 2010 and 2019. 2019 is the maximum (1,100 billion USD, lowest score -0.132). 2009 and 2010 lie in the sparse region created by the jump from 510 to 720 billion USD between them, so they are easy to isolate.

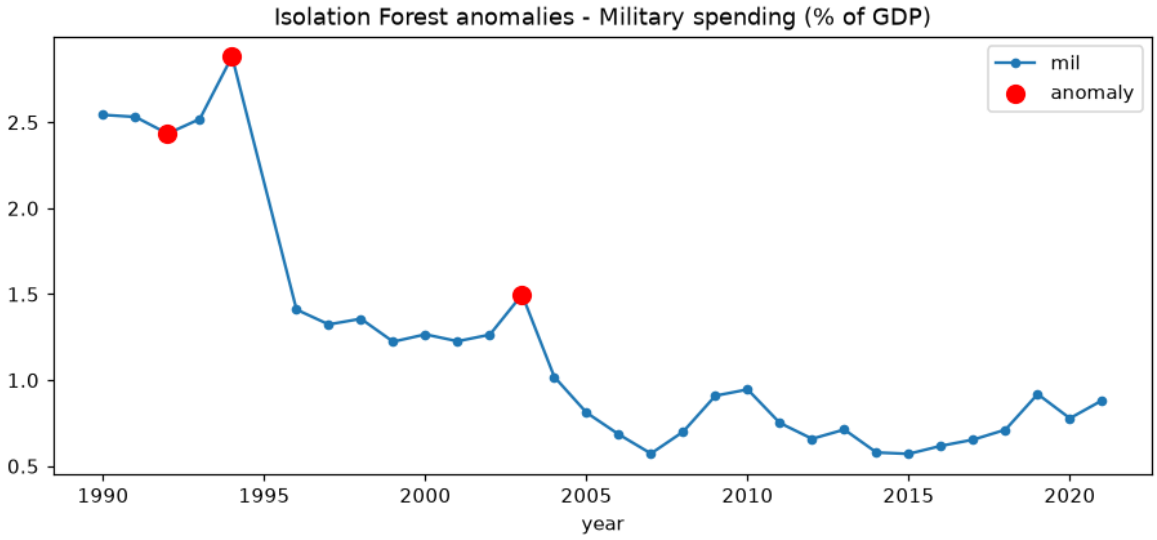
A2. Series 2: mil

Code

```
iforest_anomalies("mil")
```

Result

```
          value   score
year
1992  2.433922 -0.0892
1994  2.882670 -0.1973
2003  1.497742 -0.0282
```



Comment on the result

Flags 1994 (2.88 %, strongest, score -0.197), 1992 (2.43 %) and 2003 (1.50 %). The early 1990s are the high-spending period; 2003 is a local jump from 1.26 % (2002) to 1.50 %, the year of the military operation in Aceh.

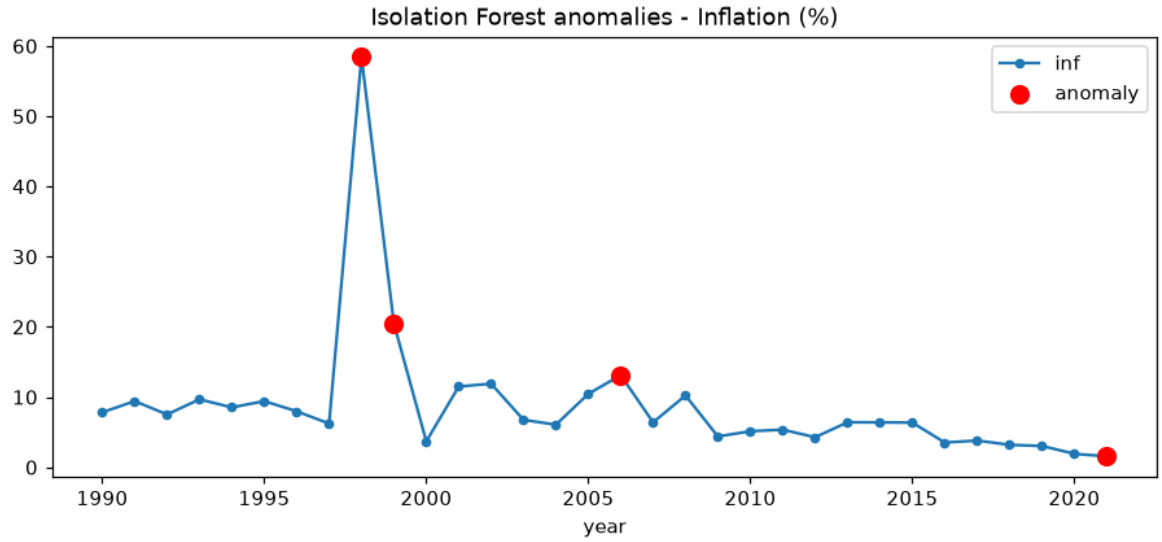
A3. Series 3: inf

Code

```
iforest_anomalies("inf")
```

Result

```
          value   score
year
1998  58.451044 -0.3054
1999  20.477831 -0.1635
2006  13.108672 -0.0018
2021   1.560130 -0.0207
```



Comment on the result

Flags 1998 (58.45 %, score -0.305, by far the strongest), 1999 (20.48 %), 2006 (13.11 %) and 2021 (1.56 %, the lowest inflation). Unlike the Z-score method, Isolation Forest also detects the 1999 after-crisis inflation, since it does not depend on the mean and standard deviation.

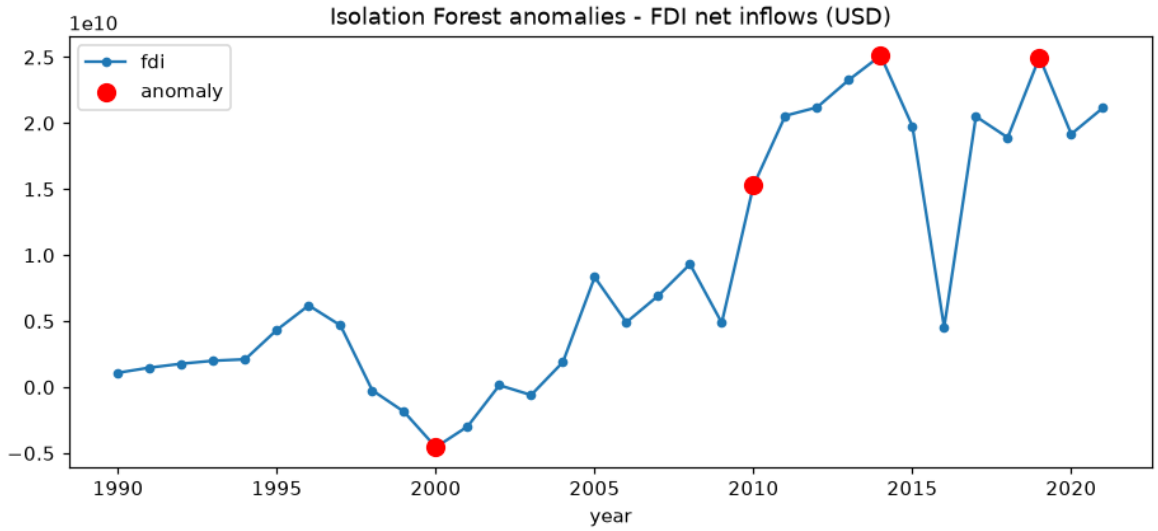
A4. Series 4: fdi

Code

```
iforest_anomalies("fdi")
```

Result

```
              value   score
year
2000 -4.550355e+09 -0.0412
2010  1.529201e+10 -0.0063
2014  2.512073e+10 -0.0086
2019  2.499355e+10 -0.0008
```



Comment on the result

Flags 2000 (-4.55 billion USD, the largest net outflow after the crisis), 2010 (15.3 billion USD, a jump from 4.9 billion in 2009), 2014 (25.1 billion, the maximum) and 2019 (25.0 billion). The scores are close to 0 (-0.04 to -0.001), so these anomalies are weak: they are mainly the extremes of the distribution.

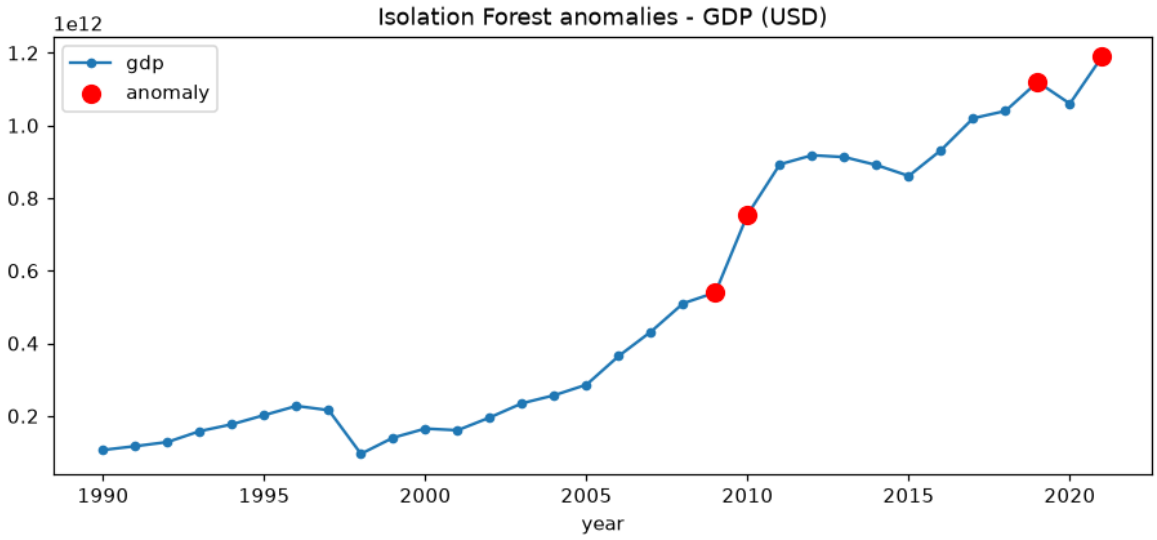
A5. Series 5: gdp

Code

```
iforest_anomalies("gdp")
```

Result

```
              value   score
year
2009   540000000000 -0.0014
2010   755000000000 -0.0221
2019  1120000000000 -0.0329
2021  1190000000000 -0.1107
```



Comment on the result

Flags 2021 (1,190 billion USD, the maximum, score -0.111), 2019, 2010 and 2009. As for green GDP, Isolation Forest on raw values flags the largest values and the years around the 2009-2010 jump. For trending series a better input would be growth rates, which would reveal the 1998 collapse.

2.3 Change Point detection

A1. Series 1: ggdp

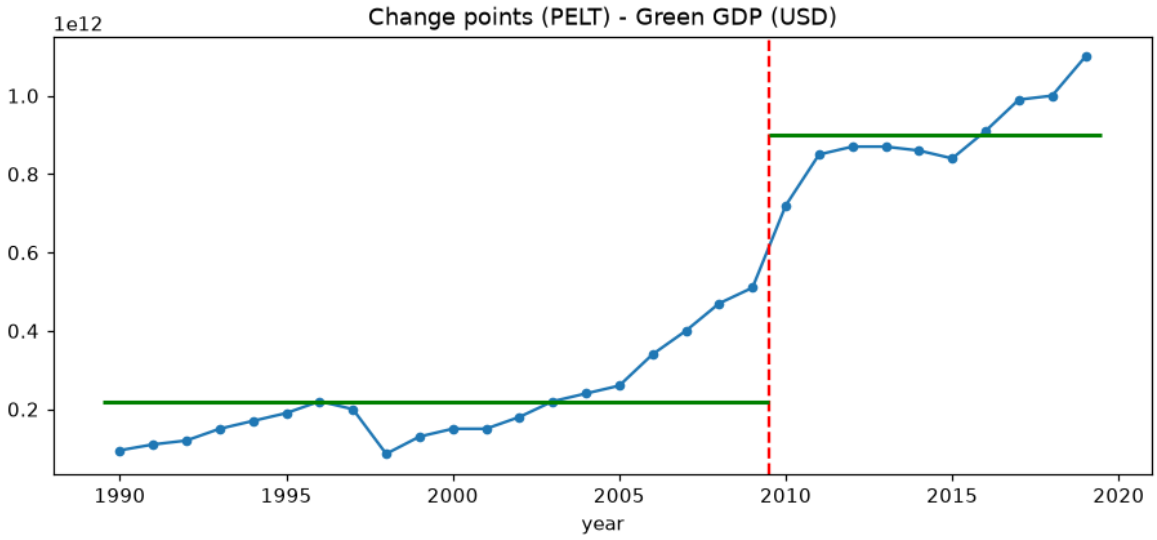
Code

```
def change_points(col, pen=3):
    s = get(col)
    x = ((s - s.mean()) / s.std()).values
    bkps = rpt.Pelt(model="rbf", min_size=3).fit(x).predict(pen=pen)
    years = [int(s.index[b]) for b in bkps[:-1]]   # first year of each new regime
    ax = s.plot(marker="o", ms=4, title=f"Change points (PELT) - {NAMES[col]}")
    for y in years:
        ax.axvline(y - 0.5, color="red", ls="--")
    for a, b in zip([0] + bkps[:-1], bkps):
        seg = s.iloc[a:b]
        ax.hlines(seg.mean(), seg.index[0] - 0.5, seg.index[-1] + 0.5, color="green", lw=2)
    plt.show()
    print("Change points (first year of new regime):", years)
    for a, b in zip([0] + bkps[:-1], bkps):
        seg = s.iloc[a:b]
        print(f"  segment {seg.index[0]}-{seg.index[-1]}: mean = {seg.mean():.4g}")

change_points("ggdp")
```

Result

```
Change points (first year of new regime): [2010]
  segment 1990-2009: mean = 2.196e+11
  segment 2010-2019: mean = 9.01e+11
```



Comment on the result

One change point in 2010: the mean green GDP rises from 220 billion USD (1990-2009) to 901 billion USD (2010-2019). The level shift corresponds to the jump from 510 to 720 billion USD in 2010, reflecting fast growth after the 2008-2009 global financial crisis, the commodity boom and the appreciation of the rupiah.

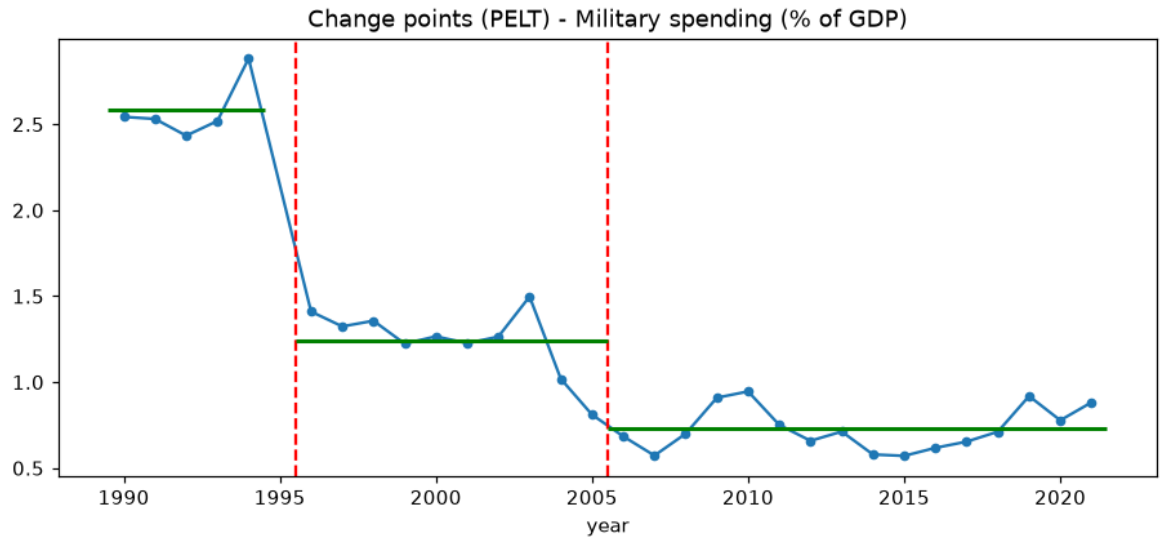
A2. Series 2: mil

Code

```
change_points("mil")
```

Result

```
Change points (first year of new regime): [1996, 2006]
  segment 1990-1994: mean = 2.582
  segment 1996-2005: mean = 1.24
  segment 2006-2021: mean = 0.7279
```



Comment on the result

Two change points, in 1996 and 2006, give three regimes: 2.58 % of GDP (1990-1994), 1.24 % (1996-2005) and 0.73 % (2006-2021). Military spending as a share of GDP fell in steps: the economy grew much faster than the defence budget.

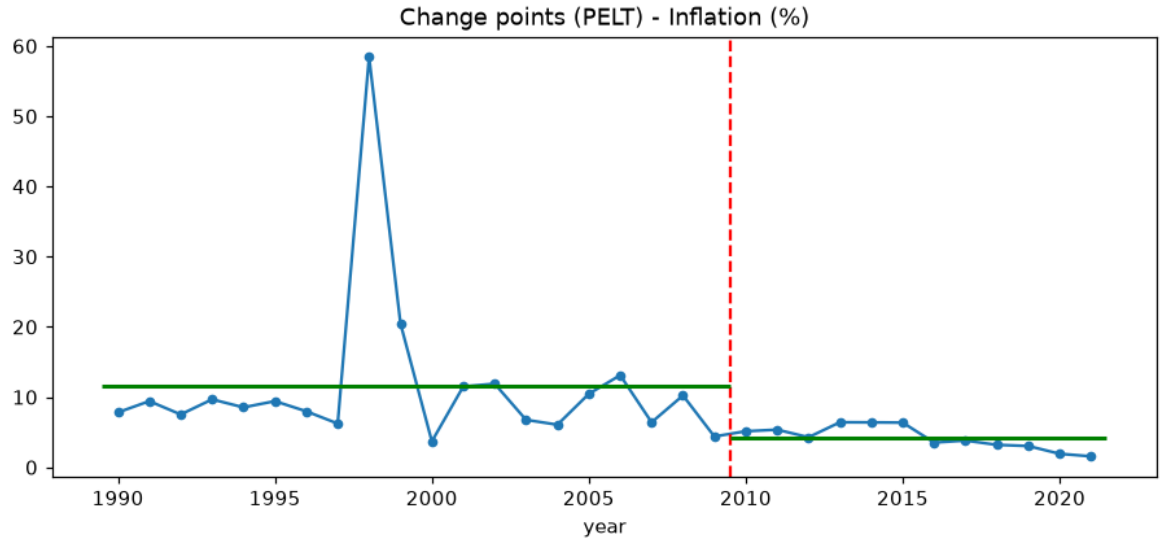
A3. Series 3: inf

Code

```
change_points("inf")
```

Result

```
Change points (first year of new regime): [2010]
  segment 1990-2009: mean = 11.5
  segment 2010-2021: mean = 4.249
```



Comment on the result

One change point in 2010: mean inflation falls from 11.5 % (1990-2009) to 4.2 % (2010-2021), a move to a low and stable inflation regime under Bank Indonesia's inflation-targeting framework. The 1998 spike is not a change point: it is a single-year anomaly and inflation returned to its previous level, which illustrates the difference between an anomaly and a change point.

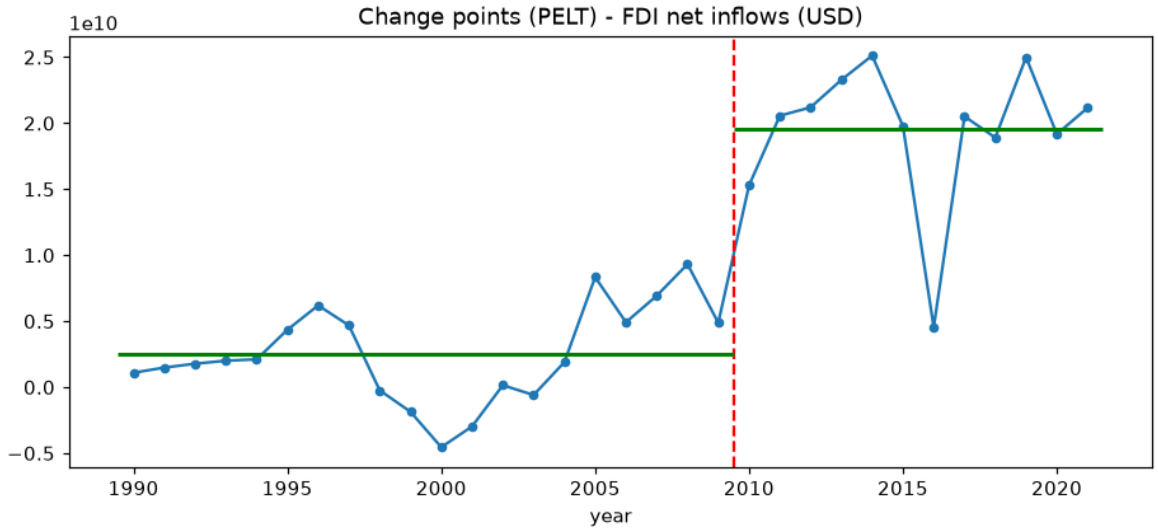
A4. Series 4: fdi

Code

```
change_points("fdi")
```

Result

```
Change points (first year of new regime): [2010]
  segment 1990-2009: mean = 2.493e+09
  segment 2010-2021: mean = 1.954e+10
```



Comment on the result

One change point in 2010: mean FDI inflows rise from 2.5 billion USD (1990-2009, including the negative years after the crisis) to 19.5 billion USD (2010-2021). Indonesia became a major destination of foreign investment after 2010.

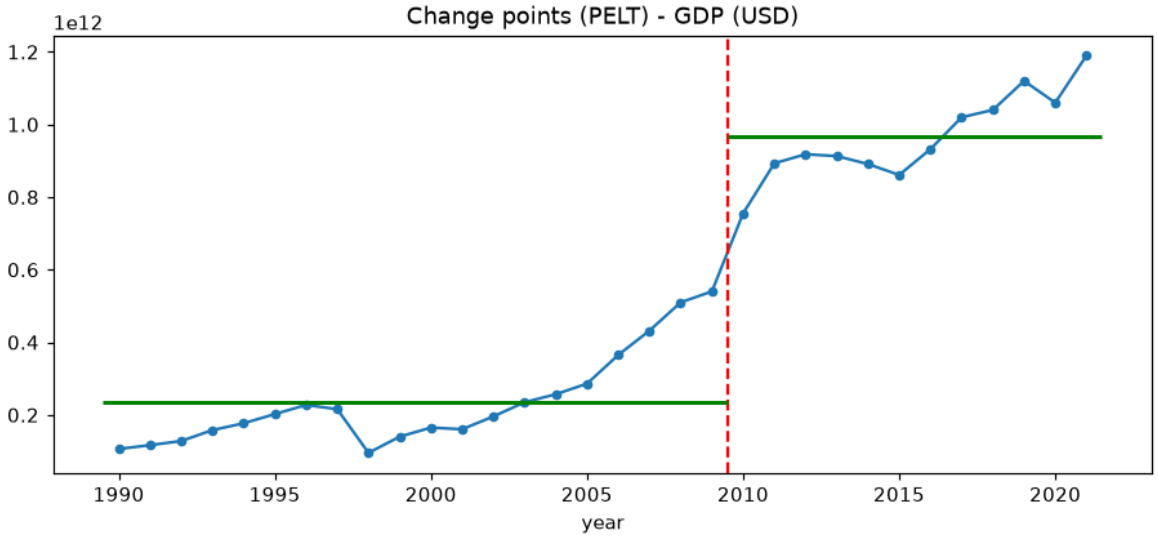
A5. Series 5: gdp

Code

```
change_points("gdp")
```

Result

```
Change points (first year of new regime): [2010]
  segment 1990-2009: mean = 2.356e+11
  segment 2010-2021: mean = 9.661e+11
```



Comment on the result

One change point in 2010: the mean GDP rises from 236 billion USD (1990-2009) to 966 billion USD (2010-2021). The same break year as green GDP, FDI and inflation suggests a common structural change in the Indonesian economy around 2010.

Part 2 conclusion

For Indonesia 1990-2021, the Z-score method ($|z| > 3$) detects only one anomaly, the 58 % inflation of 1998 (Asian financial crisis); it is not suitable for trending series such as GDP, green GDP and FDI and suffers from masking. Isolation Forest is more flexible: it detects 1998-1999 inflation, the 2000 FDI outflow and the high military spending of 1992-1994, but on raw trending data it mostly flags the extreme (latest) values and its number of anomalies is fixed by the contamination parameter. Change point detection with PELT shows a common structural break in 2010 for GDP, green GDP, FDI (level up) and inflation (level down), and a step-wise decline of military spending in 1996 and 2006. Anomaly detection and change point detection are complementary: the 1998 crisis is a short shock (anomaly), while 2010 marks a lasting change of regime.

Reference

1. Box, G. E. P., Jenkins, G. M., Reinsel, G. C. and Ljung, G. M. (2015). Time Series Analysis: Forecasting and Control, 5th ed. Wiley.

2. Hyndman, R. J. and Athanasopoulos, G. (2021). Forecasting: Principles and Practice, 3rd ed. OTexts. https://otexts.com/fpp3/

3. Dickey, D. A. and Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. Journal of the American Statistical Association, 74(366), 427-431.

4. Kwiatkowski, D., Phillips, P. C. B., Schmidt, P. and Shin, Y. (1992). Testing the null hypothesis of stationarity against the alternative of a unit root. Journal of Econometrics, 54, 159-178.

5. Liu, F. T., Ting, K. M. and Zhou, Z.-H. (2008). Isolation Forest. Proceedings of the IEEE International Conference on Data Mining, 413-422.

6. Killick, R., Fearnhead, P. and Eckley, I. A. (2012). Optimal detection of changepoints with a linear computational cost. Journal of the American Statistical Association, 107(500), 1590-1598.

7. Truong, C., Oudre, L. and Vayatis, N. (2020). Selective review of offline change point detection methods. Signal Processing, 167. (ruptures library)

8. Seabold, S. and Perktold, J. (2010). statsmodels: Econometric and statistical modeling with Python. Proceedings of the 9th Python in Science Conference.

9. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

10. General Statistics Office of Vietnam. Consumer Price Index methodology and monthly CPI releases. https://www.gso.gov.vn

11. World Bank. World Development Indicators. https://databank.worldbank.org

12. Course notes: Time Series Analysis - Machine Learning with Python, USTH ICT Department, 2026.